

UNIVERSITATEA TEHNICĂ „Gheorghe Asachi” din IAȘI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
MASTER: SECURITATEA SPATIULUI CIBERNETIC

Protocol Oauth folosind Schnorr

LUCRARE DE DISERTATIE

Coordonator științific
Ș.l.dr.inf. Silviu Paval

Absolvent
Matei Rareș

Iași, 2026

DECLARAȚIE DE ASUMARE A AUTENTICITĂȚII
LUCRĂRII DE DIPLOMĂ

Subsemnatul(a) MATEI RAREȘ,
legitimat(ă) cu CI , seria NZ , nr. 037153 , CNP 5010223271540
autorul lucrării

PROTOCOL OAUTH FOLOSIND SCHNORR

elaborată în vederea susținerii examenului de finalizare a studiilor de master, programul de studii SECURITATEA SPATIULUI CIBERNETIC organizat de către Facultatea de Automatică și Calculatoare din cadrul Universității Tehnice „Gheorghe Asachi” din Iași, sesiunea IUNIE 2026 a anului universitar 2024-2026 , luând în considerare conținutul Art. 34 din Codul de etică universitară al Universității Tehnice „Gheorghe Asachi” din Iași (Manualul Procedurilor, UTI.POM.02 – Funcționarea Comisiei de etică universitară), declar pe proprie răspundere, că această lucrare este rezultatul propriei activități intelectuale, nu conține porțiuni plagiate, iar sursele bibliografice au fost folosite cu respectarea legislației române (legea 8/1996) și a convențiilor internaționale privind drepturile de autor.

Data
03.06.2026

Semnătura



Iași, 2024

Cuprins

Introducere	1
Capitolul I. Cerințe funcționale, concepte, arhitectură.....	3
I.1. Noțiuni teoretice	3
I.1.1. Zero Knowledge Proof.....	3
I.1.2. Schema de identificare Schnorr	4
I.1.3. Open Authorization (OAuth)	5
I.2. Tehnologii utilizate	5
I.2.1. Python.....	5
I.2.2. Javascript	6
I.3. Cerinte functionale	6
I.3.1. Actorii Sistemului	6
I.3.2. Cerinte functionale	7
I.3.3. Constrangeri de securitate	7
I.4. Arhitectura aplicației	8
I.4.1. Nivelul de prezentare	8
I.4.2. Nivel de aplicație.....	8
I.4.3. Nivel de date	8
Capitolul II. Funcționalitate și implementare	10
II.1. Modelarea fluxurilor de date	10
II.1.1. Inregistrarea.....	10
II.1.2. Autentificarea	11
II.1.3. Consumul tokenului.....	15
II.2. Structura și serializarea mesajelor în protocolul HTTP	16
II.2.1. Antete si Conventii REST	16
II.2.2. Structura mesajelor	16
II.2.3. Semantica și Definirea Formală în Notăție ABNF	17
II.2.4. Gestionarea Stării	17
II.3. Detalii de implementare	18
II.3.1. Serverul de autentificare	18
II.3.2. Clientul Web.....	18
II.3.3. Derivarea cheii private: model teoretic si compromisuri de implementare	18
Capitolul III. Rezultate si validari.....	19
III.1. Strategia si metodologia de testare	19
III.2. Validarea functionala si teste de securitate	19
III.3. Evaluarea performantei	20
III.4. Simularea vectorilor de atac	24
Concluzii	26

Bibliografie.....	28
Anexe	31

Protocol Oauth folosind Schnorr

Matei Rareș

Rezumat

În prezent, aplicațiile web moderne impun mecanisme robuste de autentificare la distanță, prin care utilizatorul trebuie să-și demonstreze identitatea digitală pentru a obține acces la resurse protejate. Într-un model tradițional, acest proces se bazează pe transmiterea unui secret partajat către server, abordare care, deși protejată aparent de protocoale precum TLS, rămâne vulnerabilă în fața unor vectori de atac precum compromiterea bazelor de date, interceptarea traficului în vederea decriptării ulterioare sau atacurile de tip replay. Aceste riscuri pot conduce la furt de identitate, pierderi financiare și încălcarea confidențialității utilizatorilor.

Dificultatea de a asigura o autentificare sigură fără a expune materialul secret pe rețea, conjugată cu creșterea constantă a suprafeței de atac în ecosistemul web, a motivat proiectarea unui protocol care să elimine aceste neajunsuri structurale, propunând totodată metode concrete de consolidare a încrederii utilizatorului în propriul sistem de autentificare.

Alegerea acestei teme este susținută de necesitatea de a reduce riscurile și costurile asociate eventualelor breșe de securitate, prin utilizarea unor tehnici criptografice avansate bazate pe demonstrații cu cunoștințe zero (eng. Zero-Knowledge Proof). În plus, s-a urmărit explorarea integrării schemei de identificare Schnorr într-o soluție practică, în care utilizatorul își poate dovedi identitatea fără ca parola sau cheia privată să părăsească vreodată dispozitivul local.

Acest lucru s-a realizat prin implementarea unui protocol hibrid care imbină demonstrația criptografică Schnorr cu cadrul de autorizare delegată OAuth, asigurând o verificare matematică riguroasă a identității clientului fără partajarea de secrete prin rețea. Accentul lucrării este plasat exclusiv pe etapa de autentificare, respectiv pe demonstrarea criptografică a identității prin schema Schnorr, în timp ce componenta de autorizare, reprezentată de emiterea și consumarea jetonului JWT, este integrată strict în scop demonstrativ, pentru a ilustra viabilitatea protocolului într-un flux web complet. În aceeași logică, evaluarea comparativă cu implementările OAuth 2.0 clasice vizează diferențele de expunere a credențialelor în faza de autentificare, nu mecanismele de delegare a accesului.

În introducere, este descrisă în detaliu tematica, abordând contextul și importanța acesteia, după care sunt analizate critic soluțiile existente de autentificare și vulnerabilitățile pe care le prezintă.

Capitolul 1 oferă o prezentare a fundamentelor teoretice ZKP, schema Schnorr, OAuth, PKCE și JWT alături de motivele din spatele alegerii tehnologiilor utilizate, explicând cum aceste decizii au contribuit la dezvoltarea arhitecturii protocolului de autentificare.

Capitolul 2 se concentrează pe implementarea protocolului, oferind o descriere amănunțită a fiecărui pas din procesul de dezvoltare, a modelului matematic subiacent și a interfeței de programare a aplicațiilor (API), incluzând și componenta experimentală de evaluare comparativă

cu fluxurile OAuth clasice.

La final, în secțiunea dedicată concluziilor sunt evidențiate rezultatele obținute prin testarea automatizată și evaluarea de performanță, diverse modalități de îmbunătățire a soluției actuale, urmată de bibliografia care include referințele utilizate.

Tehnologii principale folosite, alese pentru o implementare fluidă și predispusă dezvoltării sunt următoarele: Python [1], Flask [2], SQLite [3] și JavaScript [4]. Pe partea de server, au fost utilizate biblioteci criptografice precum cryptography și sympy, iar pytest și Locust pentru evaluarea calității. Suplimentar, s-au implementat trei fluxuri OAuth 2.0 auxiliare (inclusiv o variantă PKCE și una bazată pe Authlib) ca bază de referință obiectivă pentru analiza comparativă.

Din punct de vedere software, pentru a executa programul este nevoie de versiunea 3.10 de Python împreună cu diverse biblioteci criptografice, un mediu de rulare pentru serverul web, Npm [5] și un browser modern compatibil cu standardele actuale pentru interfața clientului.

Din punct de vedere hardware, aplicația a fost testată pe un dispozitiv cu sistem de operare Windows 10, 16 GB RAM și un procesor AMD Ryzen 5 4600H.

Introducere

Aplicațiile web moderne depind în mod inerent de mecanisme robuste de autentificare la distanță, prin care se impune demonstrarea legitimității unei identități digitale de către utilizator, etapă urmată de decizia serverului privind acordarea accesului solicitat. În mod tradițional, acest proces se bazează pe utilizarea unui secret partajat, precum o parolă sau un cod temporar. Cu toate că protecția la nivelul stratului de transport prin intermediul protocolului HTTPS [6] reduce semnificativ probabilitatea interceptării datelor în tranzit, arhitectura clasică rămâne vulnerabilă în fața unei clase extinse de vectori de atac. Printre aceste vulnerabilități structurale se numără compromiterea bazelor de date, reutilizarea parolelor, configurarea defectuoasă a infrastructurii, capturarea traficului de rețea în vederea decriptării ulterioare [7], [9], precum și atacurile de tip replay asupra unor materiale de autentificare care nu sunt ancorate corespunzător în contextul sesiunii curente.

În acest context, standardul OAuth 2.0 a fost adoptat la scară largă ca mecanism principal pentru autorizarea delegată [10], [11]. Cu toate acestea, cadrul de lucru menționat nu soluționează în mod intrinsec problema atestării identității utilizatorului fără a presupune transmiterea credențialelor [14]. În majoritatea implementărilor actuale, faza de autentificare inițială delegată serverului de autorizare se bazează în continuare pe un mecanism clasic, expus riscului de exfiltrare a parolei [15]. Astfel, se conturează necesitatea imperativă de a integra o metodă de autentificare superioară din punct de vedere criptografic în etapa premergătoare emiterii jetonului de acces.

Prin urmare, obiectivul principal al acestei lucrări este reprezentat de proiectarea și fundamentarea teoretică a unui sistem de securitate în care demonstrarea identității se realizează printr-o schemă criptografică Schnorr, fundamentată pe demonstrații cu cunoștințe zero (Zero-Knowledge Proof - ZKP). Principiul arhitectural de bază impune ca parola, sau cheia privată derivată din aceasta, să nu părăsească în niciun moment perimetrul securizat al dispozitivului clientului. În această paradigmă, serverul stochează exclusiv valoarea publică asociată identității și validează riguros dovada matematică, fără a dispune de informații referitoare la secretul original. Emiterea unui jeton JWT în urma validării criptografice este integrată strict în scop demonstrativ, pentru a ilustra viabilitatea inserției protocolului Schnorr într-un flux web complet, și nu constituie un obiectiv de cercetare în sine.

Pe lângă implementarea fluxului ZKP, proiectul integrează o componentă experimentală complexă, menită să faciliteze o evaluare comparativă de profunzime. Astfel, au fost dezvoltate suplimentar două fluxuri OAuth 2.0 personalizate, incluzând o variantă bazată pe extensia PKCE și o versiune simplificată, precum și o a treia implementare generată prin intermediul bibliotecii Authlib. Aceste module secundare nu sunt destinate înlocuirii protocolului principal Schnorr, ci au rolul de a oferi o bază de referință obiectivă pentru analizarea costurilor operaționale, a structurii mesajelor tranzacționate și a gradului de expunere a credențialelor la nivelul rețelei. În aceeași

logică, evaluarea comparativă cu aceste implementări OAuth 2.0 este concentrată exclusiv pe faza de autentificare, vizând diferențele de expunere a materialului secret în tranzit, nu mecanismele de delegare a accesului ulterioare emiterii jetonului.

În plan practic, demersul științific își propune să clarifice validitatea realizării unei autentificări web complet funcționale în absența transmiterii parolei către server, precum și viabilitatea integrării acestui mecanism într-un model de autorizare pe bază de jetoane, compatibil la nivel conceptual cu ecosistemul OAuth. De asemenea, sunt evaluate în mod critic avantajele de securitate pe care protocolul propus le aduce în comparație cu soluțiile convenționale, cuantificându-se simultan costurile computaționale introduse. Nu în ultimul rând, sunt identificate limitările arhitecturii curente și sunt propuse modificările structurale necesare pentru o eventuală tranziție către un mediu de producție.

Pentru a atinge aceste deziderate, documentația este structurată în trei secțiuni principale. În primul capitol sunt introduse fundamentele teoretice, tehnologiile utilizate și arhitectura generală a sistemului. Al doilea capitol este dedicat descrierii cerințelor funcționale, prezentării modelului matematic subiacent și detalierii implementării concrete a protocolului, inclusiv a interfeței de programare a aplicațiilor (API). În cele din urmă, al treilea capitol expune metodologia de testare și rezultatele experimentale obținute, oferind o analiză de ansamblu a avantajelor și limitărilor soluției implementate, alături de direcțiile viitoare de cercetare și dezvoltare..

Capitolul I. Cerințe funcționale, concepte, arhitectură

În acest capitol sunt prezentate fundamentele teoretice ale protocolului propus, tehnologiile software alese pentru implementare, cerințele funcționale și nefuncționale ale sistemului, actorii implicați și arhitectura pe trei niveluri.

I.1. Noțiuni teoretice

I.1.1. Zero Knowledge Proof

Demonstrația cu cunoștințe zero (Zero-Knowledge Proof - ZKP), formalizată inițial de Goldwasser, Micali și Rackoff în lucrarea fundamentală privind complexitatea cunoașterii în sistemele de demonstrații interactive [16], reprezintă un protocol criptografic fundamental prin intermediul căruia o entitate, denumită solicitant (prover / doveditor), poate demonstra unei alte entități, denumită verificador (verifier), veridicitatea unei afirmații sau cunoașterea unui secret, fără a dezvălui nicio informație suplimentară dincolo de simpla atestare a adevărului. Sistemul nu urmărește să afle parola utilizatorului, ci doar să obțină certitudinea matematică a cunoașterii acesteia. Pe lângă aplicabilitatea în autentificarea standard client-server, protocoalele ZKP au devenit instrumente esențiale pentru obținerea Identității Auto-Suverane (Self-Sovereign Identity) [17] și a sistemelor care protejează confidențialitatea, spectrul de utilizare extinzându-se semnificativ în ultimii ani, de la verificarea tranzacțiilor blockchain până la validarea conformității datelor fără divulgarea conținutului [19], [20]. Această paradigmă modernă elimină necesitatea unor entități intermediare de încredere (precum furnizorii de identitate de tip Google Sign-In sau Facebook) pentru atestarea identității, permițând utilizatorului să demonstreze direct și prin rigoare matematică faptul că este eligibil pentru accesarea resurselor, păstrând controlul absolut asupra secretelor sale.

În contextul securității cibernetice, principiul de bază al tehnologiei ZKP și avantajul său conceptual major impun ca, în niciun moment al procesului de autentificare, parola sau cheia privată să nu fie transmisă prin rețea, nici în format brut, nici măcar sub formă criptată. Această proprietate rezolvă vulnerabilități critice inerente protocoalelor tradiționale. Într-un sistem tradițional, chiar dacă parola este trimisă printr-un tunel securizat (HTTPS), serverul primește un material sensibil și trebuie să îl protejeze, fiind vulnerabil în cazul compromiterii canalului de transport.

În arhitectura ZKP, rețeaua transportă exclusiv transcrisul unei dovezi (valori matematice efemere), care nu poate fi reutilizat în afara contextului în care a fost generat. Astfel, protocolul oferă o reziliență absolută la atacurile de interceptare a traficului de tip „Store Now, Decrypt Later” [23], [25], [26]; dacă un adversar observă schimbul de mesaje, acesta va capta doar numere asociate unui proces tranzitoriu, extragerea secretului fiind imposibilă matematic. În al doilea rând, previne atacurile de tip Replay, deoarece natura interactivă a protocolului presupune emiterea unei provocări unice (challenge) de către server la fiecare încercare de conectare. Totodată, atenuează complet impactul breșelor de date (Data Leaks) prin eliminarea „secretului partajat”, serverul stocând exclusiv chei publice care sunt inutile unui atacator în lipsa dispozitivului și a parolei clientului.

În cadrul acestui proiect, tehnologia ZKP are un rol arhitectural vital, fiind utilizată pentru a consolida și înlocui mecanismele vulnerabile din fluxul standard OAuth 2.0. Conceptul nu rulează în izolare, ci se realizează printr-o mapare directă a fazelor ZKP peste etapele de autorizare delegată OAuth [27]. Abordări complementare, precum integrarea schemelor zk-SNARK în mecanisme de autentificare bazate pe blockchain [28], confirmă viabilitatea utilizării

demonstrațiilor cu cunoștințe zero ca substituent al modelelor tradiționale de validare a identității, în contexte arhitecturale diverse. Concret, demonstrația ZKP, implementată în acest sistem prin schema de identificare Schnorr, preia rolul parametrului clasic de validare (precum client secret sau transmiterea parolei brute), oferind o garanție matematică a identității pentru emiterea tokenului de acces [29], [30], fără ca secretul utilizatorului să părăsească vreodată mediul local al aplicației client.

1.1.2. Schema de identificare Schnorr

Schema Schnorr [31], [33], [36] reprezintă una dintre cele mai consacrate și robuste construcții criptografice fundamentate pe dificultatea computațională a problemei logaritmului discret în grupuri finite [37]. În cadrul acestei arhitecturi, fundamentul matematic este riguros definit prin utilizarea unui număr prim sigur (safe prime), notat cu P , ce satisface egalitatea $P = 2 \cdot Q + 1$, unde Q constituie, la randul său, un număr prim de dimensiuni mari. Peste acest număr prim se consideră grupul multiplicativ $Z_P = \{1, 2, \dots, P-1\}$. Elementul central al schemei îl reprezintă alegerea unui generator $G \in Z_P$ asociat subgrupului de ordin Q , determinat prin relația matematică $G = h^2 \bmod P$, impunându-se condițiile stricte de securitate ca $G \neq 1$ și $G^Q \bmod P = 1$, pentru o valoare aleatoare $h \in Z_{P^*}$.

Protocolul se desfășoară între două entități: solicitantul (prover), care deține secretul, și verficatorul (verifier), care validează dovada fără a obține informații despre secret. Interacțiunea presupune o fază de pregătire și patru etape secvențiale:

Generarea perechii de chei: Cheia privată este reprezentată de o valoare secretă $x \in Z_Q$, cunoscută exclusiv de către solicitant. Cheia publică este dedusă matematic sub forma $y = G^x \bmod P$ și este comunicată verficatorului, care o stochează și o asociază identității solicitantului.

Etapă de angajament (Commitment): Solicitantul alege un nonce aleatoriu și efemer $r \in Z_Q$. Pe baza acestuia, se calculează angajamentul criptografic temporar $t = G^r \bmod P$, valoare transmisă verficatorului.

Generarea provocării (Challenge): Verficatorul alege aleatoriu o provocare $c \in Z_Q$ și o transmite solicitantului. Caracterul aleatoriu al provocării este esențial pentru securitatea protocolului, deoarece garantează faptul că solicitantul nu poate precalcuła un răspuns valid fără cunoașterea efectivă a cheii private [38], [39].

Calculul răspunsului (Response): Dispunând de provocarea primită, solicitantul calculează dovada matematică sub forma $s = (r + c \cdot x) \bmod Q$, valoare ce aparține grupului Z_Q . Această dovadă atestă cunoașterea secretului x fără a-l dezvălui.

Faza de verificare (Verification): Verficatorul evaluează concomitent doi termeni distincți, utilizând exclusiv cheia publică y :

$$\text{left} = G^s \bmod P$$

$$\text{right} = t \cdot y^c \bmod P$$

Autentificarea este considerată validă dacă și numai dacă egalitatea fundamentală $\text{left} = \text{right}$ este satisfăcută. Corectitudinea matematică a acestei verificări rezultă din substituția directă: $G^s = G^{(r + c \cdot x)} = G^r \cdot G^{(c \cdot x)} = t \cdot y^c \bmod P$. Această proprietate oferă garanția matematică a identității solicitantului, eliminând necesitatea schimbului sau expunerii unor secrete prin intermediul canalului de comunicare [41].

1.1.3. Open Authorization (OAuth)

Cadrul de autorizare delegată OAuth 2.0 reprezintă un standard industrial care permite aplicațiilor să obțină acces securizat la resurse protejate fără transmiterea credențialelor direct către aplicația consumatoare. Pentru clienții publici, extensia PKCE (Proof Key for Code Exchange), definită prin specificația RFC 7636 [28], adaugă un strat suplimentar de protecție împotriva interceptării codului de autorizare. În arhitectura prezentului proiect, acest cadru furnizează modelul structural pe care se grefează protocolul Schnorr: un Server de Autentificare verifică identitatea clientului și emite jetoane de acces (OAuth Access Tokens) în urma unei validări reușite.

În urma verificării criptografice, serverul generează un Access Token sub forma unui JSON Web Token (JWT) semnat, standardizat conform RFC 7519 [27], cu valabilitate limitată în timp. Clientul atașează acest jeton ca Bearer Token în antetul HTTP al cererilor ulterioare, eliminând necesitatea reluării procesului ZKP la fiecare interacțiune cu resursele protejate. Validarea jetonului este stateless: serverul verifică semnătura JWT fără a re-executa protocolul Schnorr, asigurând un cost operațional redus, conform modelului familiar aplicațiilor web moderne.

Implementările tradiționale ale fluxurilor OAuth 2.0, inclusiv variantele PKCE sau fluxul simplu Authorization Code, prezintă însă o deficiență fundamentală la nivelul fazei de autentificare. În aceste scheme, parola sau secretul brut traversează rețeaua în prima etapă, fiind transmise către serverul de autorizare. Chiar dacă materialul sensibil nu ajunge mai departe la serverul de resurse, el rămâne expus față de emitent și constituie o țintă viabilă pentru atacatorii care interceptează traficul prin tehnici de tip traffic sniffing.

Soluția propusă în această lucrare nu înlocuiește ecosistemul bazat pe jetoane, ci fortifică exact etapa cea mai vulnerabilă: validarea identității inițiale. Mecanismul convențional de autentificare a clientului (parametrul client_secret sau transmiterea parolei brute) este substituit cu o dovadă criptografică Zero-Knowledge implementată prin schema Schnorr. Fazele protocolului se mapează direct peste fluxul OAuth: angajamentul (Commitment) servește drept inițiere a cererii de acces (Grant Initiation), provocarea (Challenge) funcționează ca nonce de sesiune, iar răspunsul matematic (Response) acționează ca etapă de Client Authentication, finalizându-se cu emiterea JWT-ului. Securitatea este consolidată prin legarea de canal (Session Binding), care ancorează jetonul emis de contextul HTTP unic al sesiunii ZKP, împiedicând transferul malițios al jetonului între contexte de rețea diferite.

1.2. Tehnologii utilizate

În acest subcapitol sunt prezentate limbajele de programare, cadrele de lucru și bibliotecile utilizate în implementarea protocolului, motivând alegerea fiecărei componente în raport cu cerințele de securitate și de prototipare rapidă ale sistemului.

1.2.1. Python

Nucleul aplicației server este dezvoltat în Python, un limbaj interpretat de nivel înalt, selectat pentru claritatea sintaxei, viteza de prototipare și ecosistemul extins de biblioteci. Python constituie fundamentul serverului de autentificare, al logicii criptografice Zero-Knowledge Proof (ZKP) și al infrastructurii de testare automatizată. Arhitectura software se bazează pe următoarele biblioteci și cadre de lucru:

Flask v3.0.0 și Flask-CORS v4.0.0 [44]: Micro-framework pentru aplicații REST care gestionează rutarea URL, parsarea cererilor HTTP și serializarea răspunsurilor JSON. Flask-CORS aplică politicile de partajare a resurselor (Cross-Origin Resource Sharing) necesare

comunicării cu aplicația client web.

Flask-SQLAlchemy v3.1.1 [45]: Asigură persistența datelor printr-un ORM (Object-Relational Mapping) peste o bază de date SQLite locală (auth.db), adecvată unui prototip de laborator prin simplificarea instalării și resetarea rapidă a stării în timpul testelor. Schema integrează trei tabele: utilizatorii înregistrați (stocând cheile publice secret_y), token-urile emise (AuthToken) și entitățile de date protejate (PersoData). PyJWT v2.12.1 [46]: Creează, semnează și verifică token-uri conform standardului RFC 7519 (JSON Web Token). După validarea demonstrației Schnorr ZKP, serverul emite un Access Token semnat HS256, delegând autorizarea ulterioară fără reluarea procesului criptografic.

Authlib [47]: Furnizează o implementare de referință a unui server OAuth 2.0 conform specificațiilor PKCE, cu rol strict analitic, permițând comparația de performanță și securitate între fluxurile tradiționale de autorizare și paradigma ZKP.

pytest v9.0.3 și Locust [48], [49]: Formează nucleul ecosistemului de asigurare a calității. pytest gestionează suita de testare automatizată, acoperind cazuri funcționale pozitive, negative, scenarii limită și vectori de atac criptografici. Locust evaluează încărcarea concurentă, scalabilitatea și debitul de procesare (throughput) prin simularea unui număr mare de utilizatori.

cryptography v45.0.1 și sympy [50], [51]: Oferă primitive criptografice și instrumente matematice pentru generarea, validarea și testarea parametrilor grupului criptografic, asigurând rigurozitatea numerelor prime sigure (safe primes) utilizate ca fundament al protocolului Schnorr.

1.2.2. Javascript

Componenta client a sistemului este construită ca o aplicație web statică, utilizând JavaScript, HTML și CSS fără cadre de lucru externe, pentru a păstra codul simplu și a permite observarea directă a mecanismelor de securitate implicate. JavaScript a fost ales deoarece reprezintă limbajul standard pentru dezvoltarea interfețelor grafice web, fiind suportat nativ de toate browserele moderne fără a necesita instalarea unor componente suplimentare.

JavaScript gestionează interfața grafică (formulare, cereri asincrone către server, actualizarea stării) și, totodată, execută calculele criptografice ale protocolului ZKP pe dispozitivul utilizatorului. Prin utilizarea tipului de date BigInt pentru aritmetica de precizie arbitrară și a API-ului Web Crypto pentru generarea de valori aleatorii criptografic sigure, toate operațiile matematice ale schemei Schnorr (generarea angajamentului, calculul răspunsului) se desfășoară local. Această abordare asigură faptul că parola sau cheia privată nu parasesc browserul și nu sunt transmise prin rețea în niciun moment al procesului de autentificare.

În plus, codul include un mecanism de afișare a traficului HTTP, care permite utilizatorului să vizualizeze cererile și răspunsurile schimbate între client și server. Această funcționalitate are un rol didactic și experimental, facilitând compararea directă a mesajelor protocolului.

1.3. Cerințe funcționale

1.3.1. Actorii Sistemului

Sistemul implică trei roluri logice distincte, dintre care unele sunt colocată în cadrul prototipului pe același server:

- Clientul (Solicitant / Prover): Reprezentat de aplicația web rulată în browserul utilizatorului. Acesta deține local parola (din care se derivează cheia privată) și execută calculele matematice ale schemei Schnorr pentru a-și demonstra identitatea fără a

transmite secretul.

- Serverul de Autentificare (Verificator / Emitent de Token): Stochează exclusiv cheia publică a utilizatorului, generează provocarea de sesiune, verifică dovada matematică primită de la client și emite jetoanele de acces (JWT).
- Serverul de Resurse: Componenta logică, în prototip găzduită pe același server, care permite accesul la datele protejate strict pe baza jetonului emis în urma autentificării.

În scop experimental, arhitectura include și un Server de Autorizare OAuth 2.0, implementat în două variante personalizate și o variantă bazată pe biblioteca Authlib, utilizat pentru evaluarea comparativă a fluxurilor de securitate.

1.3.2. Definirea fluxului de autentificare

Pentru a asigura o validare criptografică robustă și o integrare fluidă cu mecanismele de autorizare delegată, sistemul trebuie să respecte următorul flux operațional:

Pre-schimbul parametrilor publici (Handshake - eng.): Sistemul trebuie să permită clientului obținerea parametrilor criptografici globali ai grupului (P, G) printr-un punct terminal (endpoint) dedicat. Această abordare previne atacurile de tip Logjam; prin utilizarea unor parametri specifici fiecărui server, se evită vulnerabilitatea majoră în fața unui pre-calcule comun (utilizând algoritmi precum Number Field Sieve) realizat de un adversar cu resurse computaționale mari asupra unui grup standardizat comun.

Înregistrarea: Sistemul trebuie să permită clientului generarea locală a perechii de chei și transmiterea exclusivă a valorii publice ($y = G^x \bmod P$) alături de un identificator ('client_id'), serverul stocând doar această asociere, fără ca parola să părăsească dispozitivul clientului.

Autentificarea, unde, protocolul trebuie să se desfășoare în următoarele etape:

- Angajament (Commitment - eng.): Clientul calculează un angajament criptografic efemer ($t = G^r \bmod P$) și îl transmite serverului împreună cu client_id, fără a expune parola.
- Provocarea (Challenge - eng.): Serverul generează un număr aleatoriu unic, asociat sesiunii curente și contextului de rețea, pe care îl transmite clientului.
- Soluția (Response - eng.): Clientul calculează și transmite dovada matematică (s) utilizând secretul propriu, angajamentul inițial și provocarea primită.
- Verificarea (Verification - eng.): Serverul validează matematic dovada prin egalitatea $G^s = t \cdot y^c \bmod P$. Dacă ecuația este satisfăcută, identitatea clientului este confirmată.

Autorizarea și Sesiunea: În urma verificării, serverul emite un jeton de acces JWT semnat, cu valabilitate limitată în timp. Clientul utilizează acest jeton (Bearer Token) pentru accesul ulterior la resurse, fără a relua protocolul.

1.3.3. Constrângeri de securitate

Securitatea și robustețea sistemului sunt asigurate prin respectarea unor constrângeri stricte de proiectare:

- Zero-Knowledge și reziliență la interceptare (Anti-Sniffing): În niciun moment al protocolului, parola sau cheia privată nu traversează rețeaua, nici măcar în format criptat. Sistemul permite autentificarea sigură chiar și pe canale nesigure sau compromise (deși criptarea de transport HTTPS adaugă un nivel de securitate suplimentar, nu este o precondiție pentru protejarea secretului).
- Protecție la atacuri tip replay și legarea de canalul de comunicare (Session Binding):

Transcrisul unei autentificări interceptate nu poate fi refolosit într-o altă sesiune. Sistemul garantează că un jeton emis este criptografic legat de sesiunea ZKP care l-a generat, prevenind transferul jetonului între contexte de rețea diferite.

- Separarea resurselor (Statelessness): Serverul trebuie să valideze jetonul JWT exclusiv prin verificarea semnăturii, fără a re-executa protocolul Schnorr.
- Validarea parametrilor: Valorile publice recepționate de la client trebuie validate matematic ca membri legitimi ai subgrupului corect. Cererile invalide, datele malformate și tentativele de fraudă trebuie respinse fără a genera erori de server.
- Transparență și Lipsa Anonimizării Identității: Sistemul permite auditarea traficului și evaluarea comparativă a protocoalelor. Identificatorul client_id este transmis în clar în fazele de înregistrare și angajament. Protocolul protejează exclusiv secretul de autentificare, nu și metadatele de identitate.

1.4. Arhitectura aplicației

Sistemul este organizat pe trei niveluri funcționale: prezentare, aplicație și date. Această separare permite delimitarea clară a responsabilităților fiecărui strat și decuplarea logicii de securitate de restul componentelor. Comunicarea dintre niveluri se realizează prin cereri HTTP asincrone, iar baza criptografică a întregului sistem este problema logaritmului discret pe grupuri finite, implementată prin schema de identificare Schnorr. Succesiunea etapelor prin care clientul și serverul interacționează pe parcursul înregistrării și autentificării a fost descrisă în secțiunea I.3.2, iar detalierea acestora la nivel de implementare, însoțită de diagramele de secvență corespunzătoare, se regăsește în secțiunea II.1.

1.4.1. Nivelul de prezentare

Nivelul de prezentare se ocupă de interacțiunea cu utilizatorul, afișarea datelor și execuția calculelor locale. Construit cu tehnologii web standard, acest nivel ghidează utilizatorul prin trei etape: afișarea ecranului introductiv, generarea perechii de chei criptografice și transmiterea valorii publice în cadrul înregistrării, respectiv execuția protocolului cu cunoștințe zero în cadrul autentificării. Tot la acest nivel este integrat un monitor de rețea în timp real, care permite utilizatorului să inspecteze traficul HTTP.

1.4.2. Nivel de aplicație

Nivelul de aplicație conține logica de procesare și gestionează punctele de acces HTTP. Acest strat, scris în Python cu ajutorul micro-framework-ului Flask, verifică dacă valorile criptografice primite aparțin subgrupului Schnorr, administrează fluxurile de autorizare și păstrează starea temporară a sesiunilor între etapele protocolului, folosind un sistem de stocare volatil. După o verificare reușită, serverul emite un jeton de acces JWT, compus din antet, sarcină utilă și semnătură criptografică. Un aspect important al acestui nivel este legarea de canal (Session Binding): serverul nu verifică dovezile izolat, ci le leagă de contextul HTTP curent, care include adresa de rețea, antetul User-Agent, identificatorul de sesiune și identitatea clientului, prevenind astfel atacurile de interceptare și retransmisie (relay și session hijacking).

1.4.3. Nivel de date

Nivelul de date asigură stocarea informației printr-o bază de date relațională, accesată prin abstractizări ORM. Baza de date păstrează trei categorii de informații: identitatea publică a

utilizatorului, referințele jetoanelor emise și datele personale protejate. Aspectul cel mai relevant al acestui nivel este modul în care sunt tratate credențialele: serverul nu stochează parola în clar, nici sub formă de hash și nici scalarul privat asociat. Singura valoare criptografică păstrată pentru verificare este cheia publică.

Capitolul II. Funcționalitate și implementare

Acest capitol detaliază implementarea protocolului de autentificare descris în capitolul anterior. Sunt prezentate fluxurile de înregistrare, autentificare și consum al jetonului JWT prin diagrame de secvență, formalizarea matematică a schemei Schnorr și fragmentele de cod relevante.

II.1. Modelarea fluxurilor de date

II.1.1. Inregistrarea

Fluxul de înregistrare reprezintă prima interacțiune a unui utilizator nou cu platforma și constituie etapa de provizionare a identității criptografice în sistem. Procesul este inițiat din interfața grafică, unde utilizatorul introduce un identificator unic (`client_id`) și o parolă, fără ca aceasta din urmă să fie transmisă către server în niciun moment al fluxului.

Aplicația client solicită de la server parametrii criptografici ai grupului Schnorr printr-o cerere `GET /parameters`, la care serverul răspunde cu valorile P și G (codul de stare HTTP 200). Pe baza acestora, clientul derivă local cheia privată x prin aplicarea funcției `sCrypt` asupra parolei și a identificatorului, rezultatul fiind redus modular la ordinul subgrupului Q , apoi calculează cheia publică $y = G^x \bmod P$. Întreaga operație criptografică se desfășoară exclusiv în browserul utilizatorului, garantând proprietatea de cunoștințe zero a protocolului. După finalizarea calculelor, clientul transmite către server o cerere `POST /register` conținând exclusiv perechea (`client_id`, `secret: y`).

La recepția cererii, serverul execută trei verificări succesive înainte de a persista datele. Mai întâi, se validează completitudinea parametrilor, iar în cazul absenței câmpului `client_id` sau `secret_y` cererea este respinsă cu codul de stare HTTP 400. Ulterior, serverul verifică apartenența valorii publice la subgrupul de ordin Q prin evaluarea condițiilor $1 < y < P$ și $y^Q \equiv 1 \bmod P$, respingând cu codul HTTP 422 orice valoare criptografică neconformă care ar putea compromite securitatea verificărilor ulterioare. În final, se interoghează baza de date pentru a determina unicitatea identificatorului: dacă acesta nu există, se inserează înregistrarea și serverul returnează codul HTTP 201, iar dacă este deja asociat unui cont existent, cererea este respinsă cu codul HTTP 409, prevenind suprascrierea silențioasă a cheii publice.

La finalizarea cu succes a procesului, interfața grafică afișează un mesaj de confirmare. Din acest moment, serverul stochează exclusiv cheia publică y asociată identificadorului, fără a deține vreo informație referitoare la parola sau la cheia privată a utilizatorului.

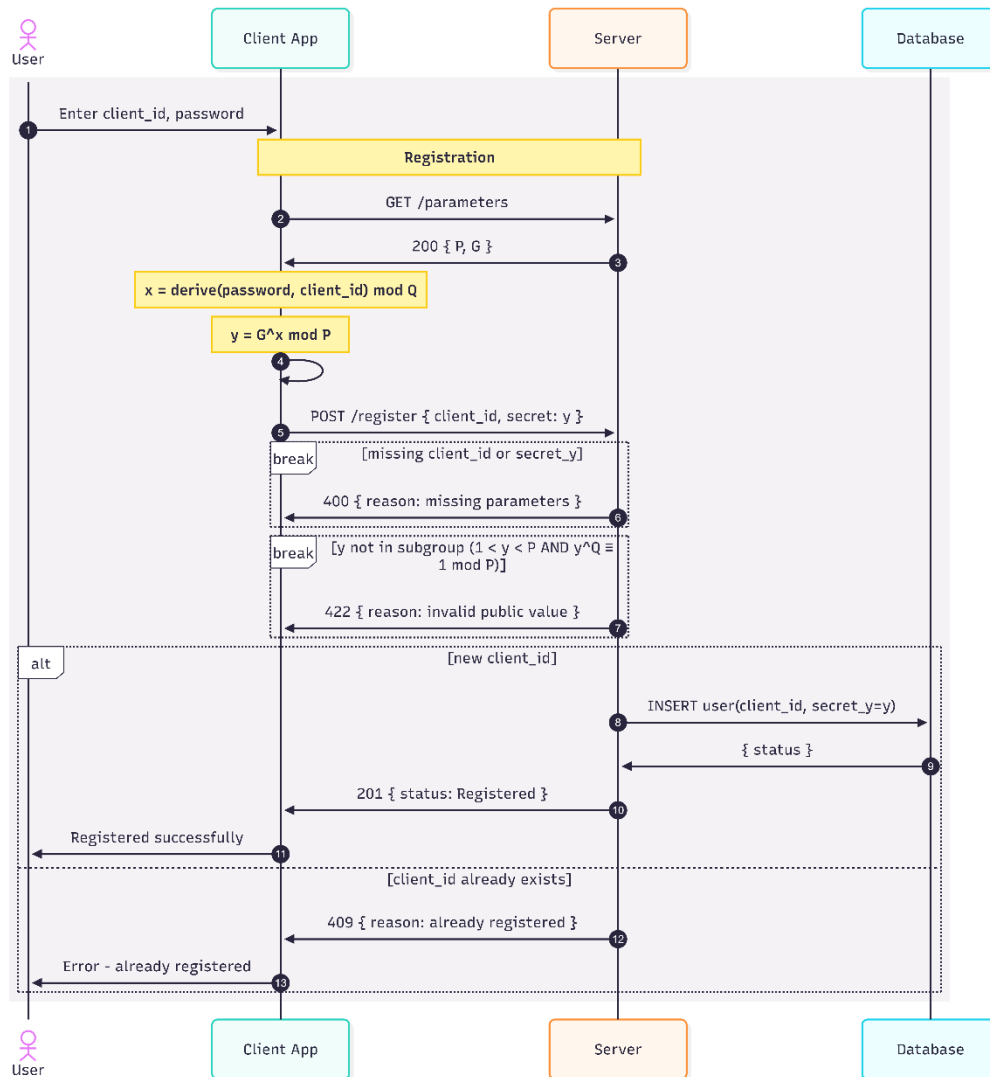


Diagrama 1: Fluxul de înregistrare

II.1.2. Autentificarea

Fluxul de autentificare implementează schema de identificare Schnorr în două etape distincte, fiecare corespunzând unei cereri HTTP separate: faza de angajament (commit) și faza de verificare (verify). Procesul este inițiat din interfața grafică, unde utilizatorul introduce identificadorul și parola, iar aplicația client derivă local cheia privată prin aceeași funcție script utilizată la înregistrare. Înainte de a detalia pașii protocolului, sunt definite în continuare fundamentele matematice pe care se sprijină întregul flux.

În faza de angajament, clientul solicită parametrii criptografici publici prin cererea GET /parameters, apoi generează un nonce aleatoriu utilizând generatorul criptografic nativ al browserului (`window.crypto.getRandomValues`) și calculează angajamentul criptografic efemer conform ecuației definite în secțiunea II.2.1. Aceste valori sunt transmise prin cererea POST /login/commit conținând identificadorul client_id și angajamentul commitment_t.

La recepția cererii, serverul verifică prezența parametrilor obligatori, respingând cu codul HTTP 400 cererile incomplete. Ulterior, se validează apartenența angajamentului la subgrupul de ordin Q , cererile cu valori neconforme fiind respinse cu codul HTTP 422. Serverul interoghează apoi baza de date pentru a confirma existența utilizatorului, returnând codul HTTP 404 în cazul unui identificator neinregistrat. Dacă toate verificările sunt satisfăcute, serverul generează un identificator unic de sesiune (`session_id`) prin intermediul unui generator criptografic de numere pseudoaleatoare (CSPRNG), calculează valoarea de legare a sesiunii (session binding) conform ecuației (2) din secțiunea II.2.1, apoi derivă provocarea pe baza acestei valori. Starea temporară a sesiunii, conținând angajamentul, provocarea, valoarea de legare, adresa de rețea și marca temporală, este persistată în memoria volatilă, iar serverul răspunde clientului cu perechea (`session_id`, `challenge_c`).

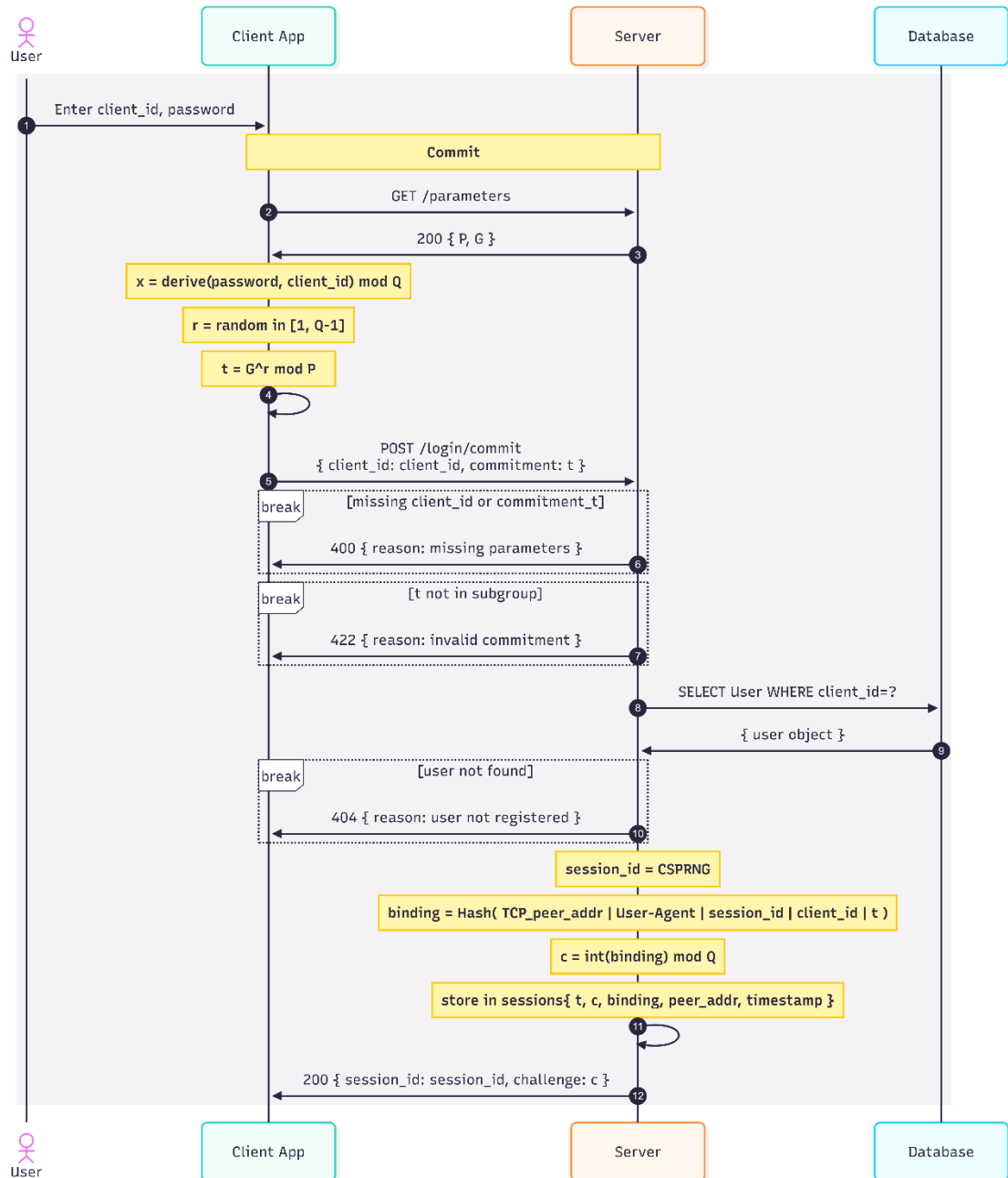


Diagrama 2: Fluxul de commit (Sequence Diagram)

Spre deosebire de standardul Schnorr convențional, prezenta soluție tehnologică derivă provocarea printr-un mecanism riguros de legare a sesiunii (session binding), ancorând transcrisul criptografic de contextul rețelei observat la nivel de server:

$$c = \text{int}(\text{SHA-256}(\text{Addr} \mid \text{UA} \mid \text{SID} \mid \text{CID} \mid t)) \bmod Q \quad (2)$$

Addr: adresa TCP a partenerului de rețea UA: agentul utilizator (User-Agent) SID: identificatorul unic și temporar al sesiunii CID: identificatorul asociat clientului t: angajamentul criptografic recepționat anterior

În faza de verificare, clientul calculează dovada matematică conform ecuației de răspuns definite în secțiunea II.2.1 și o transmite prin cererea POST /login/verify, incluzând valoarea `solution_s` în corpul JSON și identificatorul de sesiune în antetul X-Auth-Session.

La recepția cererii, serverul execută o succesiune de verificări de securitate. Se validează existența sesiunii asociate identificatorului transmis, respingând cu codul HTTP 404 sesiunile inexistente. Se verifică dacă sesiunea nu a depășit intervalul de valabilitate (TTL de 5 secunde), sesiunile expirate fiind șterse și respinse cu codul HTTP 401. Se controlează dacă valoarea răspunsului se încadrează în intervalul valid admis matematic, respingând cu codul HTTP 422 soluțiile în afara domeniului. Serverul recalculează apoi valoarea de legare a sesiunii pe baza contextului HTTP curent și o compară cu cea stocată la momentul angajamentului, respingând cu codul HTTP 401 orice neconcordanță, mecanism care previne atacurile de interceptare și retransmisie (relay și session hijacking). După parcurgerea tuturor verificărilor preliminare, serverul citește cheia publică a utilizatorului din baza de date și evaluează egalitatea Schnorr conform ecuației (1) din secțiunea II.2.1. Dacă dovada este validă, serverul înserează referința jetonului în baza de date, șterge imediat sesiunea temporară și returnează clientului jetonul JWT semnat HS256 cu codul HTTP 200. În cazul unei dovezi invalide, sesiunea este de asemenea ștearsă, iar serverul returnează codul HTTP 401.

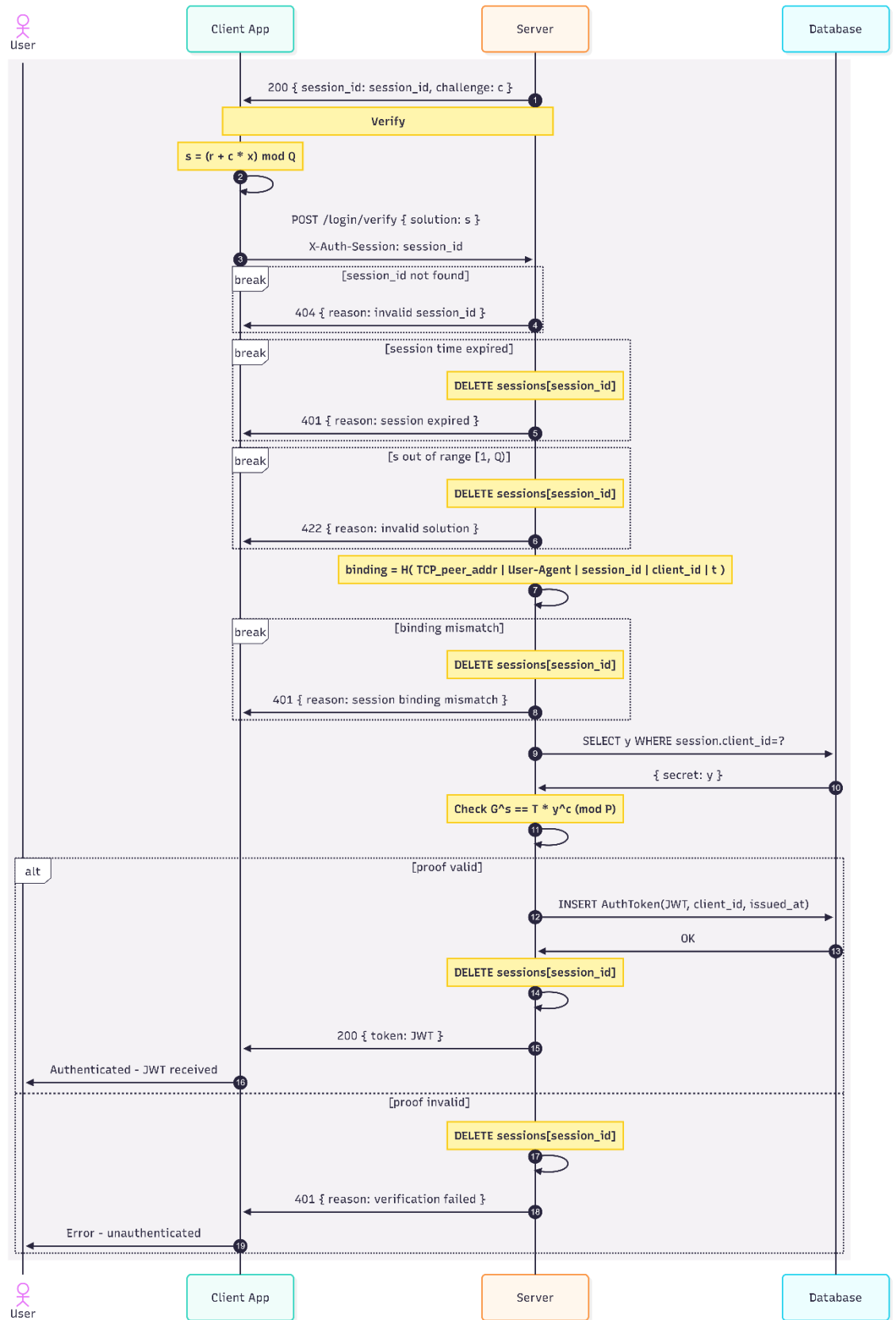


Diagrama 3: Fluxul de verificare (Sequence Diagram)

În implementarea curentă, o a doua tentativă de angajament pentru același utilizator,

apărută după o fereastră de 50 ms, este tratată ca potențială tentativă de preluare abuzivă a sesiunii și determină invalidarea sesiunii existente. Acest comportament este acoperit de testele automate, însă reprezintă și un compromis de ergonomie care este discutat în secțiunea dedicată limitărilor.

Arhitectura implementată se fundamentează pe utilizarea unui prim sigur (safe prime) P , garantând astfel un număr prim $Q = (P - 1) / 2$. Prin stabilirea generatorului $G = 4$, operațiunile matematice se desfășoară exclusiv în subgrupul de ordin Q asociat lui Z_{P^*} . Componentele esențiale ale schemei sunt definite după cum urmează:

Cheia privată: x Cheia publică: $y = G^x \bmod P$ Valoarea efemeră (nonce): r Angajamentul (commitment): $t = G^r \bmod P$ Provocarea (challenge): c Răspunsul: $s = (r + c \cdot x) \bmod Q$

Se observă că validarea identității este confirmată exclusiv prin satisfacerea următoarei egalități:

$$G^s \bmod P = t \cdot y^c \bmod P \quad (1)$$

s : răspunsul matematic calculat local de către client t : angajamentul criptografic inițial y : cheia publică stocată a utilizatorului c : provocarea matematică verificată de server..

II.1.3. Consumul tokenului

După finalizarea autentificării, clientul utilizează jetonul JWT obținut ca Bearer Token în antetul Authorization al cererilor către resursele protejate. Serverul verifică semnătura HMAC-SHA256 și validitatea câmpurilor standard (exp, iss, aud), permițând accesul cu codul HTTP 200 sau respingând cererea cu codul HTTP 401 în cazul unui jeton invalid ori expirat. Fluxul ZKP implementat nu include un mecanism de reînnoire a jetonului, astfel încât la expirarea intervalului de valabilitate clientul trebuie să reia integral protocolul de autentificare.

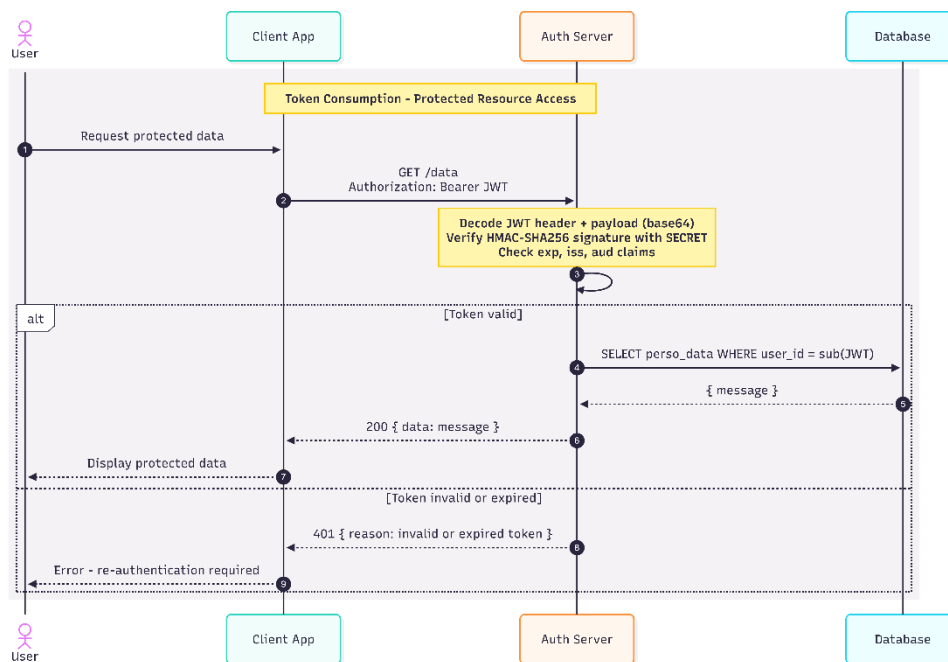


Diagrama 4: Consumul Tokenului (Post-Authentificare)

II.2. Structura și serializarea mesajelor în protocolul HTTP

Pentru a asigura interoperabilitatea și o comunicare deterministă între client (client – engl.) și server (server – engl.), protocolul definește formal schema de date, antetele necesare și formatul mesajelor transmise [53].

II.2.1. Antete si Conventii REST

Interfața expusă este de tip REST JSON (Representational State Transfer – engl.), necesitând utilizarea tipului de conținut „application/json”. Pe lângă antetele standard de autorizare, implementarea emite o serie de antete suplimentare pentru trasabilitate și securitate, precum „Request-ID”, „API-Version”, „X-Response-Time”, „Server-Timing”, „X-Content-Type-Options”, „X-Frame-Options”, „Content-Security-Policy” și „Referrer-Policy” [54]. În cazul răspunsurilor neautorizate de tip 401, este inclus și antetul „WWW-Authenticate”. Punctele terminale (endpoints – engl.) principale ale arhitecturii sunt definite după cum urmează:

- /health (GET): Verificarea stării de funcționare a serverului.
- /parameters (GET): Publicarea parametrilor criptografici globali P și G.
- /register (POST): Înregistrarea identificadorului și a valorii publice a secretului.
- /login/commit (POST): Inițierea autentificării prin transmiterea angajamentului criptografic.
- /login/verify (POST): Verificarea dovezii matematice și emiterea jetonului.
- /data (GET, POST, PUT): Accesarea resurselor protejate exclusiv prin jeton.
- /oauth/pkce/și /oauth/simple/(POST): Fluxuri de autorizare comparativă (Authorization Code – engl.) [10].

II.2.2. Structura mesajelor

Valorile numerice de magnitudine extinsă, rezultate din calculele efectuate asupra grupului Schnorr, sunt serializate sub formă de șiruri de caractere (strings) reprezentând numere întregi în baza zece. Această convenție de codificare este impusă de absența unui tip nativ pentru numere întregi de precizie arbitrară în specificația JSON, precum și de limitările de precizie ale tipului IEEE 754 disponibil în mediul de execuție JavaScript al clientului web [4]. Pornind de la această convenție, structura sarcinii utile este detaliată în continuare pentru fiecare etapă a protocolului.

În etapa de înregistrare, clientul transmite identificadorul unic al utilizatorului și cheia publică derivată local:

```
{ "client_id": "alice", "secret_y": "12345678901234567890" }
```

Odată ce identitatea criptografică a fost provizionată, protocolul de autentificare debutează cu etapa de angajament (commit), în care clientul transmite identificadorul și angajamentul criptografic efemer, calculat pe baza valorii aleatorii r:

```
{ "client_id": "alice", "commitment_t": "98765432109876543210" }
```

Ca răspuns la acest angajament, serverul returnează provocarea matematică și identificadorul de sesiune asociat:

```
{ "challenge_c": "112233445566778899", "session_id": "opaque-session-token" }
```

Pe baza provocării recepționate, clientul calculează dovada matematică și o transmite în

etapa de verificare (verify), atestând astfel cunoașterea cheii private fără a o expune:

```
{ "solution_s": "998877665544332211" }
```

În cazul unei validări reușite, serverul returnează un jeton de acces semnat:

```
{ "token": "jwt-string" }
```

Se constată că niciuna dintre structurile prezentate nu conține parola, cheia privată sau vreun derivat al acestora. Câmpurile vehiculate se limitează la identități publice, valori criptografice efemere și artefacte de sesiune, proprietate care decurge direct din natura protocolului cu cunoștințe zero descris în secțiunea II.1.2.

II.2.3. *Semantica și Definierea Formală în Notăție ABNF*

Fiecare variabilă implicată îndeplinește un rol fundamental în mitigarea vulnerabilităților de rețea. Provocarea generată de server invalidează atacurile de reluare (replay attacks – engl.) impunând o demonstrație temporală unică, în timp ce angajamentul asociază criptografic sesiunea de un element aleatoriu nedețvăluit. Pentru asigurarea standardizării formale, sintaxa se definește prin notația ABNF (Augmented Backus-Naur Form – engl.) [55]:

```
client-id = 1*(ALPHA / DIGIT / "-" / "*")
schnorr-value = 1*DIGIT
session-id = 1*(ALPHA / DIGIT / "-" / "*")
jwt-token = 1*(ALPHA / DIGIT / "." / "-" / "_" )
q-string = DQUOTE 1*(VCHAR / SP) DQUOTE

registration-payload = "{" DQUOTE "client_id" DQUOTE ":" q-string "," DQUOTE
"secret_y" DQUOTE ":" q-string "}"
commitment-payload = "{" DQUOTE "client_id" DQUOTE ":" q-string "," DQUOTE
"commitment_t" DQUOTE ":" q-string "}"
challenge-response = "{" DQUOTE "challenge_c" DQUOTE ":" q-string "," DQUOTE
"session_id" DQUOTE ":" q-string "}"
verify-payload = "{" DQUOTE "solution_s" DQUOTE ":" q-string "}"
auth-success-payload = "{" DQUOTE "token" DQUOTE ":" q-string "}"

authorization-header = "Bearer" SP jwt-token
session-header = "X-Auth-Session:" SP session-id
```

Din punct de vedere conceptual, se observă că protocolul limitează transferul de date la identități publice și parametri de sesiune, excluzând în totalitate expunerea cheii private [41].

II.2.4. *Gestionarea Stării*

Protocolul matematic impune retenția unei stări temporare (stateful – engl.) între momentul inițierii angajamentului și faza verificării. Pentru sistemele distribuite, este necesară externalizarea acestei stări către o memorie volatilă centralizată. Permisunile pre-solicitare sunt gestionate riguros prin strategii de partajare a resurselor între origini (Cross-Origin Resource Sharing – engl.). Odată finalizată autorizarea, se recomandă evitarea stocării jetonului în spații locale expuse vulnerabilităților de injecție a scripturilor (Cross-Site Scripting – engl.), fiind indicată încapsularea acestuia în cookie-uri protejate prin directivele „HttpOnly” și „Secure” [56].

II.3. *Detalii de implementare*

II.3.1. *Serverul de autentificare*

Arhitectura de server este construită pe micro-framework-ul Flask, care constituie nucleul logic de procesare al întregului protocol [57]. Stratul de validare criptografică se sprijină pe funcția `is_subgroup_member`, care evaluează condiția matematică $value^Q \bmod P = 1$ și respinge totodată valorile triviale, eliminând astfel riscurile asociate atacurilor de tip subgrup mic (small subgroup attack) [32]. Această verificare este invocată atât la înregistrarea cheii publice, cât și la recepția angajamentului efemer, conform fluxurilor descrise în secțiunile II.1.1. și II.1.2.

Gestionarea concurenței sesiunilor este delegată structurii `_SessionStore`, care menține un index invers de la identificatorul de utilizator la identificatorul de sesiune activă. Prin această asocierie bidirecțională, serverul detectează situațiile în care un al doilea angajament este inițiat pentru un utilizator care dispune deja de o sesiune în curs, invalidând-o automat conform politicii de protecție descrise în secțiunea II.1.2.

Legarea contextului de rețea este realizată de funcția `_compute_session_binding`, care corelează sesiunea curentă cu metadatele de transport, iar emiterea jetonului de acces revine rutinei `_issue_jwt`, ce generează un jeton JWT cu o durată de valabilitate de o oră. Accesul la resursele protejate prin punctul terminal `/data` este condiționat de validarea prealabilă a semnăturii și a expirării jetonului, în timp ce antetele de tip Cache-Control [58] previn stocarea răspunsurilor în nodurile intermediare ale rețelei.

II.3.2. *Clientul Web*

Nivelul de prezentare execută întreaga logică criptografică direct în mediul de rulare al browserului, fără a delega operațiuni sensibile către server. Implementarea este concentrată în modulul `login.js` [4], care orchestrează secvențial etapele protocolului descrise în secțiunea II.1.2.: obținerea parametrilor publici, derivarea scalarului privat, generarea valorii aleatorii r prin interfața nativă `window.crypto.getRandomValues` [59], calculul angajamentului și, ulterior, al răspunsului matematic pe baza provocării recepționate de la server. Întregul ciclu de comunicare este supravegheat de un monitor de rețea integrat în interfață, care interceptează apelurile `fetch` și redă în timp real structura completă a antetelor și a sarcinilor utile, facilitând astfel auditarea vizuală a protocolului în scopuri didactice și de depanare.

II.3.3. *Derivarea cheii private: model teoretic și compromisuri de implementare*

Derivarea cheii private x din parola utilizatorului constituie un punct critic al lanțului criptografic, deoarece robustețea întregii scheme Schnorr depinde de impredictibilitatea acestei valori. Modelul teoretic, implementat în scriptul `calculations.py`, aplică funcția `sCrypt` [60] cu un salt aleatoriu dedicat fiecărui utilizator, asigurând astfel rezistență sporită la atacuri de tip dicționar și forță brută prin consumul deliberat de memorie și timp de calcul.

În varianta demonstrativă destinată browserului, modulul `auth.js` substituie această derivare cu o rezumare SHA-256 aplicată asupra concatenării identificatorului cu parola, fără utilizarea unui salt criptografic independent. Această simplificare, adoptată din motive de portabilitate și compatibilitate cu mediul de execuție JavaScript, reduce semnificativ costul computațional al derivării, expunând însă cheia privată la atacuri precalculate (rainbow tables). Totodată, modulul client integrează un mecanism de rezervă care recurge la parametri criptografici de dimensiuni reduse ($P = 2089$, $G = 4$) în cazul indisponibilității serverului, compromis acceptabil exclusiv într-un context experimental controlat. Implicațiile acestor compromisuri asupra securității globale a sistemului sunt analizate în detaliu în secțiunea dedicată limitărilor.

Capitolul III. Rezultate si validari

În acest capitol sunt prezentate metodologia de testare, rezultatele obținute în urma rulării suitelor automate și a măsurătorilor de performanță, precum și o analiză critică a avantajelor și limitărilor protocolului propus.

III.1. Strategia si metodologia de testare

Validarea sistemului a fost realizată printr-o strategie multistratificată, acoperind exhaustiv corectitudinea funcțională, robustețea, securitatea criptografică și performanța. Ca instrument central a fost utilizat cadrul de testare automatizat pytest, integrat într-un modul de orchestrare care operează asupra unui client Flask de testare izolat de mediul HTTP. Pentru a se garanta reproductibilitatea analizei, starea bazei de date și a sesiunilor este reinițializată sistematic înaintea fiecărei instanțe, eliminând orice dependențe secvențiale între cazurile de test.

Complementar validării funcționale, au fost integrate instrumente de evaluare a performanței destinate monitorizării latenței și a consumului de memorie, simulări ale încărcării concurente prin intermediul platformei Locust, precum și un mecanism de audit al traficului HTTP și scenarii de simulare a atacurilor criptografice. Rezultatele acestor evaluări sunt detaliate în secțiunile III.3 și, respectiv, III.4.

III.2. Validarea functionala si teste de securitate

În cadrul procesului de evaluare empirică a sistemului propus, s-a procedat la analiza sistematică a cazurilor pozitive care validează comportamentul corect al platformei pe traseul nominal (happy path). Se observă că înregistrarea unui utilizator nou implică stocarea exclusivă la nivelul serverului a cheii publice definite prin relația $y = G^x \bmod P$, fără a se reține parola în formă brută sau sub aspectul unui rezumat criptografic calculabil, aspect confirmat prin verificarea explicită a faptului că înregistrarea serializată nu conține date în clar sau amprente de tip SHA-256, SHA-512 ori MD5. Fluxul complet de autentificare, structurat pe etapele de angajament (commit) și verificare (verify), a fost validat prin generarea de către client a unui element aleatoriu (nonce) notat cu r , determinarea angajamentului $t = G^r \bmod P$, recepționarea provocării c și transmiterea dovezii $s = (r + c \cdot x) \bmod Q$ către server. Serverul evaluează ulterior consistența ecuației de verificare $G^s \equiv t \cdot y^c \pmod{P}$, emite un jeton JWT și elimină instanța de sesiune pentru a împiedica reutilizarea acesteia, asigurând totodată că accesul la resursele protejate returnează un cod de stare HTTP 200 în prezența unui jeton valid, respectiv un cod HTTP 401 în cazul absenței, expirării sau invalidității acestuia, fapt ce permite funcționarea paralelă și independentă a utilizatorilor multipli fără interferențe la nivelul stării.

Evaluarea comportamentului defensiv al protocolului a impus implementarea unor scenarii negative menite să confirme respingerea controlată a tentativelor de acces neautorizat sau de fraudă electronică. În situația introducerii unei parole incorecte, se constată că dovada calculată pe baza unei valori eronate nu satisface ecuația de validare, determinând serverul să returneze codul HTTP 401 și să distrugă sesiunea utilizată pentru a bloca atacurile repetitive pe același canal. De asemenea, tentativele de retransmitere (replay attack), bazate pe refolosirea unui identificador de sesiune și a unei soluții deja procesate, determină generarea unui răspuns de tip HTTP 404 sau HTTP 401 ca urmare a eliminării imediate a stării după prima utilizare validă. Depășirea timpului de viață (TTL), setat la o fereastră de 5 secunde între etapele de angajament și verificare, conduce la invalidarea automată a cererii cu un răspuns HTTP 401, în timp ce mecanismul de prevenire a deturnării (hijacking prevention), testat prin transmisii duble de tip angajament pentru același

identificator, generează un cod de eroare HTTP 409 și anulează ambele sesiuni concurente pentru a bloca suprascrierea silențioasă, politică aplicată în mod similar și în cazul înregistrărilor duplicate.

În continuarea analizei, determinarea robusteții algoritmului la frontierele matematice ale grupului Schnorr și în prezența unor date de intrare neconforme a fost realizată prin testarea extinsă a cazurilor limită (corner cases), context în care s-a urmărit reacția sistemului la valori ale soluției situate în afara intervalului admis, acestea din urmă fiind respinse prin coduri HTTP 422. Pentru a preveni atacurile bazate pe subgrupuri mici (small subgroup attack), introducerea unor chei publice triviale aparținând setului $\{0, 1, -1, P-1\}$ sau a unor angajamente nule la faza de inițiere determină respingerea imediată a solicitărilor fără generarea de sesiuni orfane. În condiții de concurență ridicată, simularea a zece fire de execuție simultane pentru același utilizator a demonstrat stabilitatea mecanismului de blocare prin excludere reciprocă (mutex lock), lăsând activă cel mult o sesiune validă, în timp ce introducerea unor tipuri de date malformate în câmpurile numerice, cum ar fi numere cu virgulă mobilă, notații științifice, șiruri de caractere sau valori nule, este interceptată prin coduri HTTP 400 sau 422, confirmându-se totodată robustețea antetului de autentificare X-Auth-Session.

Analiza proprietăților de securitate specifice schemei de identificare Schnorr a evidențiat rezistența protocolului în fața unor vectori de atac avansați, printre care se numără verificarea legării angajamentului (commitment binding). S-a demonstrat că un adversar capabil să construiască un istoric simulat valid, definit prin corelația $t_{sim} = G^s \cdot y^{(-c)} \bmod P$ fără cunoașterea prealabilă a valorii r , se află în imposibilitatea de a reutiliza acea dovadă falsificată împotriva unei sesiuni ancorate într-un angajament real diferit, serverul respingând prin HTTP 401 orice corelație neconformă cu starea stocată. În mod corelat, tentativele de forjare a soluției (solution forgery) în absența cheii private, fie prin transmiterea elementului aleatoriu brut, fie prin alterarea biților (bit-flip) sau introducerea de valori marginale extreme, au fost sistematic invalidate prin mecanismele de verificare a domeniului matematic, securitatea fiind completată de imposibilitatea deducerii prin încercări succesive (brute-force) a identificatorilor de sesiune și de izolarea riguroasă a contextelor de lucru aparținând unor utilizatori distincți (cross-user isolation).

III.3. Evaluarea performanței

Evaluarea latenței individuale a operațiilor criptografice fundamentale din cadrul protocolului s-a realizat prin intermediul platformei Flask Test Client. Prin această abordare metodologică, s-au eliminat penalitățile de rețea și timpii asociați serializării HTTP, obținându-se o măsurătoare precisă a efortului computațional brut. Valorile rezultate, reprezentând mediile calculate pe un eșantion de 100 de iterații, sunt sintetizate în tabelul următor:

Operație criptografică	Medie (ms)	Min (ms)	Max (ms)	P95 (ms)	StdDev (ms)
Calculul angajamentului: $t = G^r \bmod P$	0,432	0,386	1,032	0,503	0,080
Verificarea Schnorr: $G^s \equiv t \cdot y^c \pmod{P}$	0,938	0,845	1,472	1,058	0,074

Operație criptografică	Medie (ms)	Min (ms)	Max (ms)	P95 (ms)	StdDev (ms)
Challenge PKCE S256 (SHA-256)	0,004	0,003	0,029	0,004	0,004
Hash check OAuth2 Simple	0,002	0,002	0,017	0,002	0,001

Din analiza datelor, se observă că factorul de cost dominant în cadrul protocolului Zero-Knowledge Proof este reprezentat de operațiile de exponențiere modulară (respectiv $G^r \bmod P$, $G^s \bmod P$ și $y^c \bmod P$). Latența totală alocată fazei de verificare a fost cuantificată la aproximativ 0,94 ms, valoare care, deși este cu cel puțin două ordine de mărime superioară latențelor specifice operațiilor de dispersie utilizate de metodele OAuth 2.0 clasice, se încadrează în limitele optime pentru garantarea unei experiențe de autentificare interactive fluide.

O analiză de granulație fină asupra operațiilor interne ale serverului a relevat costurile individuale de execuție la un nivel profund, rezultatele fiind prezentate în tabelul de mai jos:

Operație internă	Medie (ms)	Min (ms)	Max (ms)	P95 (ms)	StdDev (ms)
Validare apartenență la subgrup	0,499	0,438	0,868	0,625	0,063
Derivare legare sesiune (Session Binding)	0,002	0,001	0,019	0,002	0,002
Generare provocare matematică	0,001	0,001	0,003	0,001	0,000
Emitere jeton JWT	0,020	0,016	0,087	0,029	0,009
GET /parameters	0,247	0,195	0,856	0,497	0,100
POST /register	1,458	1,126	11,175	1,786	1,001
GET /data	1,009	0,837	2,975	1,553	0,301
POST /data	1,307	0,995	12,502	1,611	1,146

Se constată că operația criptografică secundară dominantă este validarea apartenenței la subgrup (0,499 ms), care asigură protecția împotriva atacurilor bazate pe subgrupuri mici. Funcțiile auxiliare, precum derivarea parametrului de legare a sesiunii (0,002 ms) sau generarea provocării matematice (0,001 ms), presupun eforturi computaționale neglijabile, în timp ce procesul final de emitere a jetonului JWT (0,020 ms) reprezintă o fracțiune minimală din costul total de procesare. Punctele terminale convenționale ale aplicației (GET /parameters, POST /register, GET /data, POST /data) prezintă latențe cuprinse între 0,25 ms și 1,46 ms, cu valori maxime ocazionale atribuibile presiunii garbage collector-ului sau accesului la baza de date SQLite.

Complementar evaluării operațiilor izolate, s-a procedat la măsurarea latenței complete a fluxurilor de autentificare de la un capăt la altul (end-to-end), înglobând toate etapele protocolare necesare unei autentificări reușite. Rezultatele, sintetizate în tabelul următor, reflectă costul integral perceput de un client care parcurge întreg ciclul de autentificare:

Flux de autentificare	Medie (ms)	Min (ms)	Max (ms)	P95 (ms)
ZKP flux complet (commit + verify)	3,966	3,264	14,086	4,605
OAuth2 PKCE (auth + token)	0,558	0,471	1,509	0,799
OAuth2 Simple (authorize + token)	0,599	0,476	1,885	0,944
Authlib PKCE (authorize + token)	0,889	0,771	1,448	1,231

Fluxul ZKP complet, cuprinzând cele două runde protocolare (commit și verify), înregistrează o latență medie de 3,97 ms, valoare de aproximativ 5 până la 7 ori superioară celei aferente fluxurilor OAuth 2.0. Descompunerea pe etape a ciclului ZKP la nivelul serverului relevă o contribuție de 1,38 ms pentru faza de angajament (commit) și de 2,16 ms pentru faza de verificare (verify), cu un timp total de parcurgere dus-întors (round-trip) de 3,56 ms. Diferența față de media end-to-end de 3,97 ms este atribuibilă overhead-ului de serializare și deserializare a corpurilor JSON la nivelul clientului de testare.

În vederea determinării debitului maxim de procesare, s-a instrumentat un benchmark comparativ care a cuantificat numărul de fluxuri complete de autentificare executate pe secundă. Evaluarea s-a efectuat pentru dimensiuni variabile ale loturilor de utilizatori simulați secvențial, rezultatele fiind următoarele:

Utilizatori concurenți	ZKP (RPS)	OAuth2 PKCE (RPS)	OAuth2 Simple (RPS)	Authlib PKCE (RPS)
10	178,5	1488,6	1628,0	815,4
25	230,1	1666,4	1577,3	1052,0
50	227,3	1526,0	1812,3	774,5
100	264,0	1488,9	1641,1	1116,7

Protocolul ZKP manifestă un debit stabil cuprins între 178 și 264 RPS, valoare care prezintă o reziliență notabilă la creșterea gradului de concurență, fapt justificat de natura fixă a costului exponențierilor modulare executate pe grupul criptografic de 2048 biți. În contrast, metodele OAuth 2.0, fundamentate exclusiv pe derivări hash, înregistrează performanțe cantitative superioare, cuprinse între 774 și 1812 RPS. Raportul de performanță stabilit, de aproximativ 1:7 în favoarea metodelor clasice, constituie compromisul computațional necesar pentru asigurarea proprietăților de securitate zero-knowledge și eliminarea vulnerabilităților de transport ale credențialelor.

Din perspectiva managementului memoriei, evaluările au indicat o amprentă extrem de redusă a stării temporare gestionate de server. Dimensiunea superficială a structurii de sesiuni (shallow size) s-a menținut la o valoare constantă de 200 de octeți, în timp ce dimensiunea

profundă (deep size), incluzând totalitatea obiectelor referențiate, a variat între 1186 și 1190 de octeți, indiferent de volumul lotului procesat (100, 500 sau 1000 de fluxuri). Numărul de sesiuni active simultane în structura de stocare a rămas la valoarea unitară pe durata tuturor testelor controlate, confirmând eficiența mecanismului de distrugere a sesiunilor după prima utilizare. Aceste date validează viabilitatea stocării efemere în memoria aplicației pentru un mediu prototipal, necesitatea externalizării acestei stări într-o soluție de tip in-memory data store (precum Redis) fiind pertinentă exclusiv în cazul unor implementări distribuite. Consumul general al resurselor hardware, monitorizat pe o perioadă de 60 de secunde în condiții de trafic live, a înregistrat o utilizare medie a procesorului de 2,43%, cu un vârf maxim de 7,8%, și o amprentă de memorie RAM stabilă, cu o valoare medie de 62,09 MB și un maxim de 66,06 MB.

În vederea confirmării proprietății zero-knowledge la nivelul traficului de rețea, s-a efectuat un audit riguros al conținutului pachetelor HTTP transmise în fiecare paradigmă de autentificare. Această procedură de testare deține o valoare probatorie fundamentală pentru prezenta cercetare, întrucât demonstrează în mod direct ce informații sunt expuse unui potențial adversar capabil să intercepteze canalul de comunicație. Analiza punctelor terminale ale protocolului ZKP (POST /login/commit și POST /login/verify) a relevat că acestea vehiculează exclusiv valori numerice efemere, respectiv angajamentul t și dovada matematică s , ambele reprezentate ca numere întregi de mari dimensiuni, lipsite de orice corelație directă cu secretul utilizatorului. În contrast, fluxul de autentificare clasic (POST /classic/login) expune parola în format brut (plaintext) în corpul cererii, aceasta fiind complet vizibilă oricărui adversar capabil să intercepteze canalul de transport. În schemele OAuth 2.0 PKCE și Simple, deși credențialele nu sunt propagate către serverul de resurse, ele sunt transmise obligatoriu către serverul de autorizare în etapa de aprobare (POST /oauth/pkce/authorize, respectiv POST /oauth/simple/authorize), vulnerabilitatea la nivelul acestui segment al rețelei menținându-se nealterată. Implementarea bazată pe Authlib prezintă un comportament similar, parola fiind inclusă alături de parametrii PKCE în corpul cererii de autorizare (POST /authlib/oauth/authorize). Astfel, schema criptografică Schnorr integrată reprezintă singura variantă operațională în care materialul secret nu traversează rețeaua, capturarea integrală a unui schimb de mesaje ZKP neoferind unui atacator nicio informație exploatabilă pentru o autentificare frauduloasă ulterioară sau pentru extragerea credențialelor.

Validarea finală a stabilității arhitecturale a constat într-un test dinamic de sarcină, orchestrat cu ajutorul utilitarului Locust. S-a simulat un trafic concurent generat de 50 de utilizatori simultani, desfășurat pe un interval de 30 de secunde. Analiza rezultatelor agregate a confirmat execuția cu succes a 23 137 de cereri, fără înregistrarea niciunei erori de procesare, cu o rată medie agregată de aproximativ 390 de cereri pe secundă. Latența mediană per cerere s-a situat la 120 ms, cu o percentilă P95 de 3000 ms și un timp maxim de 6830 ms, variații explicabile prin concurența ridicată și mecanismele de planificare ale utilitarului. Privind fluxurile complete de autentificare reconstituite de Locust, protocolul ZKP a înregistrat o latență mediană de 340 ms (P95: 3600 ms), fluxurile OAuth2 PKCE și Simple s-au poziționat la o latență mediană de 210, respectiv 220 ms (P95: 3400 ms), iar varianta Authlib PKCE a prezentat o latență mediană de 97 ms (P95: 2600 ms). Convergența relativă a timpilor de răspuns în raport cu diferențele marcante observate în regim izolat (benchmark offline) demonstrează că, într-un mediu cu încărcare realistă, diferențele strict criptografice sunt puternic atenuate de costurile asociate infrastructurii, de alocarea resurselor de rețea și de suprasarcina utilitarului de testare, confirmând viabilitatea operațională a protocolului ZKP în condiții de producție..

III.4. Simularea vectorilor de atac

ds Pentru evaluarea exhaustivă a robusteții arhitecturii propuse, a fost elaborată și executată o campanie experimentală axată pe simularea automată a principalilor vectori de atac cibernetic, structurată pe patru direcții: compromiterea bazei de date, evaluarea calității entropiei criptografice, injectarea de parametri slabi prin interceptarea canalului de comunicație și demonstrarea ne fezabilității computaționale a problemei logaritmului discret.

În cadrul primului scenariu, s-a simulat o breșă de securitate (data breach) în care un adversar obține acces neautorizat la stratul de persistență al serverului, extrăgând cheia publică $y = G^x \bmod P$ asociată unui utilizator legitim. Încercarea ulterioară de a construi o dovadă falsificată, utilizând cheia publică extrasă în locul scalarului secret, formulată prin relația $s_{\text{attacker}} = (r + c \cdot y) \bmod Q$, a condus inevitabil la eșecul validării ecuației $G^{s_{\text{attacker}}} \equiv t \cdot y^c \pmod{P}$, serverul returnând codul de eroare HTTP 401. Această respingere este fundamentată pe inegalitatea $y \neq x$ în spațiul exponenților: cheia publică y reprezintă rezultatul exponențierii modulare, nu exponentul în sine, iar substituirea acesteia în formula dovezii produce un dezechilibru matematic imposibil de compensat fără cunoașterea valorii private x . Se confirmă astfel că sustragerea materialului criptografic stocat pe server nu furnizează unui atacator capacitatea de generare a unor dovezi valide cu cunoștințe zero, proprietate care diferențiază fundamental schema Schnorr de paradigmele bazate pe stocarea rezumatelor criptografice ale parolelor.

În completarea analizei de reziliență, s-a evaluat calitatea sursei de entropie criptografică utilizată pentru generarea provocărilor și a identificatorilor de sesiune. Printr-o secvență de 10 000 de cereri de angajament (commit) consecutive, s-a verificat distribuția statistică și absența coliziunilor atât pentru valorile provocărilor (challenge_c), cât și pentru identificatorii de sesiune (session_id). Toate cele 10 000 de valori au fost confirmate ca fiind strict unice, fără nicio coliziune detectată, iar provocările au fost corect încadrate în intervalul matematic specificat $[1, P-2]$, validându-se astfel eficacitatea generatorului de numere pseudo-aleatoare securizat criptografic (CSPRNG) implementat în arhitectură. Unicitatea provocărilor este esențială pentru prevenirea atacurilor de tip replay, întrucât reutilizarea unei valori c ar permite unui adversar care a interceptat un schimb anterior (t, c, s) să reproducă autentificarea fără a deține secretul, în timp ce unicitatea identificatorilor de sesiune, generați pe 256 de biți de entropie, face ca probabilitatea de ghicire prin forță brută să fie de ordinul $1/2^{256}$ per tentativă, fapt confirmat experimental prin 500 de încercări succesive de acces cu identificatori aleatorii, toate respinse cu codul HTTP 404.

Pe de altă parte, o suprafață de atac vulnerabilă în cadrul protocoalelor bazate pe problema logaritmului discret este reprezentată de faza de distribuție a parametrilor publici. În acest sens, s-a simulat un atac de tip Man-in-the-Middle (MitM) bazat pe injectarea unor parametri slabi ($P = 23$, $Q = 11$, $G = 4$), prin interceptarea și substituirea răspunsului punctului terminal GET /parameters. S-a demonstrat că, prin acceptarea acestor parametri minimali de către un client neprotejat, complexitatea problemei logaritmului discret colapsează, grupul criptografic având doar 11 elemente în loc de aproximativ 2^{1023} , ceea ce permite rezolvarea prin forță brută în cel mult $P - 2$ pași. În consecință, un adversar poate recupera cheia privată a victimei și poate finaliza cu succes fluxul protocolului în calitate de impostor. Cu toate acestea, testul complementar a demonstrat că serverul, atunci când operează cu parametrii originali de înaltă securitate, respinge în mod proactiv la faza de înregistrare orice cheie publică y care nu satisface condiția de apartenență la subgrupul legitim ($y^Q \bmod P = 1$), returnând codul HTTP 422 și blocând atacul înainte de a permite crearea unui cont exploatabil. Această vulnerabilitate impune constrângerea

arhitecturală ca aplicația client să ancoreze (pinning) valorile parametrilor de referință și să respingă orice deviație detectată în rețea, iar pe partea de server, validarea strictă a apartenenței la subgrup constituie o barieră eficientă împotriva injectării de parametri frauduloși.

Pentru a demonstra fezabilitatea securității arhitecturii pe termen lung, a fost realizat un experiment de escaladare progresivă a atacului prin forță brută asupra problemei logaritmului discret. Pornind de la instanțe triviale de 8 biți ($P = 167$, spațiu de căutare de 83 de candidați) și urcând treptat prin pragurile de 12, 16, 20, 24, 28 și 32 de biți, toate firele de execuție au fost lansate în paralel cu un termen limită de 600 de secunde. Rezultatele au confirmat o creștere exponențială a efortului de calcul: instanțele de 8 biți au fost rezolvate în 26 de iterații (1,8 secunde), cele de 24 de biți în 2 891 456 de iterații (3,2 secunde), iar cele de 32 de biți în 744 702 253 de iterații (282,7 secunde). La pragul de 64 de biți, atacul a fost întrerupt după epuizarea termenului de 600 de secunde, parcurgând doar 1 444 529 212 de iterații din cele aproximativ $4,6 \cdot 10^{18}$ necesare. Instanțele de 128 de biți și, respectiv, de 1024 de biți, corespunzătoare parametrilor efectivi ai protocolului, au înregistrat un comportament similar, demonstrând că spațiul de căutare de ordinul 2^{511} candidați depășește cu mult capacitatea computațională a oricărei arhitecturi clasice existente sau previzibile. Prin fundamentarea sistemului pe un număr prim sigur (safe prime) de 2048 de biți și un subgrup operațional de aproximativ 1023 de biți, obținerea cheii private prin metode exhaustive devine computațional nefezabilă în ipotezele standard de complexitate algoritmică actuale.

Concluzii

Lucrarea a urmărit proiectarea, implementarea și evaluarea unui sistem de autentificare care combină schema Schnorr de identificare cu un model de autorizare bazat pe JWT și inspirat conceptual din ecosistemul OAuth 2.0. Rezultatul este un prototip coerent, funcțional și bine instrumentat, care demonstrează că autentificarea web poate fi realizată fără transmiterea parolei către server, modificând paradigma modelului de încredere: serverul este degrevat de responsabilitatea protejării unui secret partajat, asumându-și strict rolul de verificator al unor relații matematice asimetrice.

Din punct de vedere funcțional, proiectul oferă un flux complet de înregistrare, autentificare, emitere de jeton și acces la resurse protejate, completat de măsuri experimentale de întărire precum validarea apartenenței la subgroup, legarea sesiunii la contextul HTTP, eliminarea sesiunii după consum și validarea strictă a intrărilor. Suita principală de testare, alcătuită din 56 de teste, a fost promovată integral, iar simulările automate suplimentare de atac (compromiterea bazei de date, evaluarea entropiei criptografice, injectarea de parametri slabi și escaladarea forței brute asupra logaritmului discret) s-au încheiat fără erori, confirmând consistența funcțională și experimentală a implementării. Ancorarea transcrisului criptografic de o sesiune temporară și de contextul specific rețelei neutralizează atacurile de tip replay și relay, iar integrarea jetoanelor JWT post-autentificare facilitează un acces la resurse eficient și compatibil cu standardele arhitecturale ale interfețelor API moderne. Prezența fluxurilor comparative bazate pe standardul OAuth 2.0 și disponibilitatea unui punct terminal dedicat parametrilor globali (/parameters) permit o evaluare riguroasă a compromisului inerent dintre securitate și performanță, fundamentând conceptual posibilitatea utilizării unor parametri criptografici specifici fiecărui server.

Rezultatele experimentale evidențiază clar compromisul fundamental al soluției. Fluxul ZKP complet înregistrează o latență medie de 3,97 ms și un debit cuprins între 178 și 264 RPS, valori de aproximativ 5 până la 7 ori superioare, respectiv de aproximativ 7 ori inferioare celor aferente fluxurilor OAuth 2.0. Acest cost computațional, generat de exponențierile modulare pe grupul criptografic de 2048 de biți, constituie compromisul necesar pentru eliminarea transmiterii parolei prin rețea, aspect confirmat direct de auditul conținutului traficului HTTP. Testul de sarcină, desfășurat cu 50 de utilizatori concurenți pe o perioadă de 30 de secunde (23 137 de cereri procesate fără erori), a demonstrat că într-un mediu cu încărcare realistă diferențele strict criptografice sunt puternic atenuate de costurile asociate infrastructurii, confirmând viabilitatea operațională a protocolului. Consumul de resurse hardware s-a menținut la niveluri minime, cu o utilizare medie a procesorului de 2,43% și o amprentă de memorie RAM stabilă la aproximativ 62 MB, iar amprenta de memorie a sesiunilor (1190 de octeți în adâncime, cu o singură sesiune activă simultan) validează eficiența mecanismului de distrugere după prima utilizare.

Prin stocarea exclusivă a cheii publice la nivelul serverului, vulnerabilitățile asociate potențialelor breșe de securitate la nivelul bazei de date sunt minimizate considerabil, adversarul neputând genera dovezi valide fără cunoașterea scalarului privat, chiar în ipoteza accesului complet la stratul de persistență. Campania de escaladare a forței brute a confirmat ne fezabilitatea computațională a recuperării cheii private pentru dimensiuni ale grupului de 64 de biți și peste, spațiul de căutare de ordinul 2^{511} candidați aferent parametrilor efectivi ai protocolului depășind cu mult capacitatea oricărei arhitecturi clasice existente sau previzibile.

În ciuda acestor rezultate, sistemul prezintă limitări structurale care circumscriu domeniul său de aplicabilitate. Fluxul bazat pe demonstrații cu cunoștințe zero introduce o latență de execuție superioară în comparație cu schemele clasice OAuth 2.0, iar cerința menținerii unei stări temporare între etapele de angajament și verificare adaugă un nivel de complexitate în scenariile de scalare orizontală. Transmiterea identificatorului de client (client_id) în clar invalidează asigurarea

anonimității la nivel de metadata, iar post-verificare, jetonul JWT devine unicul artefact de acces, compromiterea acestuia conferind unui atacator autorizări nelegitime până la momentul expirării. La nivelul implementării clientului, derivarea simplificată a secretului prin intermediul funcției SHA-256 nu integrează o funcție de derivare a cheilor cu consum ridicat de memorie, iar mecanismul de rezervă (fallback) către parametri criptografici de dimensiuni reduse, deși acceptabil într-un mediu controlat, devine critic într-o implementare de producție. Distribuția parametrilor publici nu este protejată prin mecanisme de semnare digitală sau fixare a certificatelor (pinning), devenind un vector de vulnerabilitate în absența securizării prin protocolul TLS. La nivelul infrastructurii serverului, stocarea în memorie a sesiunilor, politicile CORS permissive, utilizarea sistemului de gestiune SQLite, definirea statică a cheilor secrete și semnătura simetrică HS256 a jetoanelor subliniază caracterul pur experimental al aplicației. Timpul de viață al sesiunii, fixat la 5 secunde, este restrictiv pentru clienții mobili sau rețelele cu latență ridicată, în timp ce fereastra de 50 de milisecunde utilizată pentru diferențierea competiției pentru resurse de atacurile de preluare a sesiunii prezintă instabilitate operațională în afara mediului de laborator. Cu toate acestea, parametrii utilizați în prezenta iterație, precum generatorul $G = 4$ și funcția de derivare a cheilor scrypt cu un factor de cost $n = 2^{11}$, sunt adecvați exclusiv unui prototip de cercetare, iar pentru o viitoare tranziție către un mediu de producție se impune adoptarea unui generator standardizat (conform specificației RFC 3526) și utilizarea unor parametri scrypt aliniați la recomandările curente de securitate OWASP ($n \geq 2^{14}$).

Pentru a asigura maturizarea sistemului și alinierea acestuia la standardele industriale, se propun mai multe direcții strategice de optimizare. Cu prioritate, se impune standardizarea procesului de derivare a secretului prin implementarea unor algoritmi robusți (precum scrypt sau Argon2), proces dependent de un salt criptografic generat și stocat în mod securizat pe dispozitivul clientului, precum și eliminarea procedurilor de rezervă destinate parametrilor slabi și implementarea unui sistem de validare a parametrilor publici prin fixarea amprentelor criptografice. Referitor la fundamentul matematic, se recomandă tranziția către o schemă Schnorr aplicată pe curbe eliptice (ECC) pentru o eficientizare a costurilor computaționale, sau explorarea unor protocoale alternative de tip SRP [61], [62], PAKE [63] ori J-PAKE [64]. Infrastructura necesită externalizarea stocării sesiunilor către soluții de tip in-memory data store (precum Redis) și migrarea stratului de date relațional către platforme robuste, precum PostgreSQL sau MySQL. Din perspectiva gestionării jetoanelor, se preconizează trecerea de la semnături simetrice (HS256) către algoritmi asimetrice (RS256 sau ES256), decuplând procesele de emitere de cele de verificare. Consolidarea perimetrului de securitate va presupune integrarea unor mecanisme de limitare a ratei de acces (rate limiting), implementarea jurnalizării structurate a evenimentelor de securitate și a unor liste de revocare a jetoanelor, iar comunicarea trebuie restricționată exclusiv prin protocolul HTTPS, cu aplicarea unor politici CORS riguroase. Pe termen lung, extinderea arhitecturii prin încorporarea jetoanelor de tip proof-of-possession va elimina riscurile inerente modelului de acces bazat exclusiv pe transmiterea unui bearer token.

Prin urmare, concluzia principală a lucrării este că autentificarea bazată pe dovezi de cunoaștere zero poate fi integrată eficient într-o arhitectură web modernă și poate reduce semnificativ expunerea credențialelor la interceptare, cu condiția ca implementarea să fie completată ulterior printr-o întărire riguroasă a distribuției parametrilor, a derivării secretelor și a infrastructurii operaționale...

Bibliografie

- [1] ***, Python Programming Language, <https://www.python.org>, ultima accesare: 26/06/2024.
- [2] ***, Flask Framework, <https://flask.palletsprojects.com/en/3.0.x/>, ultima accesare: 30/05/2026.
- [3] ***, SQLite, <https://www.sqlite.org/>, ultima accesare: 30/05/2026.
- [4] ***, JavaScript / ECMAScript Language Specification, <https://ecma-international.org/publications-and-standards/standards/ecma-262/>, ultima accesare: 30/05/2026.
- [5] ***, Npm, <https://www.npmjs.com/>, ultima accesare: 30/05/2026.
- [6] D. Hardt, "The OAuth 2.0 Authorization Framework", RFC 6749, Internet Engineering Task Force, oct. 2012, <https://datatracker.ietf.org/doc/html/rfc6749>.
- [7] ***, OAuth 2.0, <https://oauth.net/2/>, ultima accesare: 30/05/2026.
- [8] ***, OAuth, <https://oauth.net/>, ultima accesare: 30/05/2026.
- [9] D. Hardt et al., "The OAuth 2.1 Authorization Framework (Draft)", Internet Engineering Task Force, <https://datatracker.ietf.org/doc/html/draft-ietf-oauth-v2-1>, ultima accesare: 30/05/2026.
- [10] S. Li, "A Study on the Applicability of OAuth in the E-Commerce Environment", IEEE International Conference on Service Sciences, 2013, doi: 10.1109/ICSS.2013.24, <https://ieeexplore.ieee.org/abstract/document/6625487>.
- [11] V. S. P. Farag, "An Analysis of the OAuth 2.0 Standard", IEEE Symposium on Security and Privacy Workshops, 2011, <https://ieeexplore.ieee.org/abstract/document/6123701>.
- [12] S. Goldwasser, S. Micali, C. Rackoff, "The Knowledge Complexity of Interactive Proof Systems", SIAM Journal on Computing, vol. 18, nr. 1, pp. 186-208, 1989, doi: 10.1137/0218012, https://people.csail.mit.edu/silvio/Selected%20Scientific%20Papers/Proof%20Systems/The_Knowledge_Complexity_Of_Interactive_Proof_Systems.pdf.
- [13] M. A. Ferrag, S. Manoj, "Overview and Applications of Zero Knowledge Proof (ZKP)", Academia, 2020, https://www.academia.edu/43094897/Overview_and_Applications_of_Zero_Knowledge_Proof_ZKP.
- [14] E. Ben-Sasson, I. Bentov, Y. Horesh, M. Riabzev, "Scalable, Transparent, and Post-Quantum Secure Computational Integrity (ZK-STARKs)", Cryptology ePrint Archive, Report 2018/046, <https://eprint.iacr.org/2018/46>.
- [15] A. Sakala et al., "zkAt: A Zero-Knowledge Authentication Primitive", Cryptology ePrint Archive, Report 2025/921, <https://eprint.iacr.org/2025/921>.
- [16] A. Amro, T. T. Nguyen, "Blockchain Authentication Scheme Using zk-SNARK Zero-Knowledge Proofs", Electronics, vol. 13, nr. 14, art. 2730, 2024, <https://www.mdpi.com/2079-9292/13/14/2730>.
- [17] M. Boubakri et al., "Integrating Zero-Knowledge Proofs with OAuth 2.0 for Multi-Agent Systems", E3S Web of Conferences, vol. 469, 2023, https://www.e3s-conferences.org/articles/e3sconf/abs/2023/106/e3sconf_icegc2023_00085/e3sconf_icegc2023_00085.html.
- [18] D. Terpstra, "Token-Based Authentication and Authorization Using Zero-Knowledge Proofs for Web API Security", Master's Thesis, Dakota State University, 2023, <https://scholar.dsu.edu/theses/425/>.
- [19] C. P. Schnorr, "Efficient Signature Generation by Smart Cards", Journal of Cryptology, vol. 4, nr. 3, pp. 161-174, 1991, doi: 10.1007/BF00196725, <https://link.springer.com/article/10.1007/bf00196725>.
- [20] A. J. Menezes, P. C. van Oorschot, S. A. Vanstone, "Handbook of Applied Cryptography", CRC Press, 1996, <https://cacr.uwaterloo.ca/hac/>.
- [21] J. Katz, Y. Lindell, "Introduction to Modern Cryptography", ed. 2, CRC Press, 2014.
- [22] F. Hao, "Schnorr Non-interactive Zero-Knowledge Proof", RFC 8235, Internet Engineering Task Force, sept. 2017, <https://www.rfc-editor.org/rfc/rfc8235.html>.

- [23] ***, zkDocs: Schnorr Zero-Knowledge Protocol, <https://www.zkdocs.com/docs/zkdocs/zero-knowledge-protocols/schnorr/>, ultima accesare: 30/05/2026.
- [24] ***, zkp-hmac-communication-js: JavaScript implementation of a Schnorr-style ZKP protocol combined with HMAC, <https://github.com/zk-Call/zkp-hmac-communication-js>, ultima accesare: 30/05/2026.
- [25] ***, Schnorr Signature, Wikipedia, https://en.wikipedia.org/wiki/Schnorr_signature, ultima accesare: 30/05/2026.
- [26] D. Boneh, "Applied Cryptography: Schnorr Identification Protocol", Stanford CS355, Lecture 5, 2019, <https://crypto.stanford.edu/cs355/19sp/lec5.pdf>.
- [27] M. Jones, J. Bradley, N. Sakimura, "JSON Web Token (JWT)", RFC 7519, Internet Engineering Task Force, mai 2015, <https://datatracker.ietf.org/doc/html/rfc7519>.
- [28] S. Sakimura, J. Bradley, N. Agarwal, "Proof Key for Code Exchange by OAuth Public Clients (PKCE)", RFC 7636, Internet Engineering Task Force, sept. 2015, <https://datatracker.ietf.org/doc/html/rfc7636>.
- [29] ***, Flask-CORS, <https://flask-cors.readthedocs.io/>, ultima accesare: 30/05/2026.
- [30] ***, Flask-SQLAlchemy, <https://flask-sqlalchemy.palletsprojects.com/>, ultima accesare: 30/05/2026.
- [31] ***, PyJWT, <https://pyjwt.readthedocs.io/en/stable/>, ultima accesare: 30/05/2026.
- [32] ***, Authlib, <https://authlib.org/>, ultima accesare: 30/05/2026.
- [33] ***, pytest, <https://docs.pytest.org/>, ultima accesare: 30/05/2026.
- [34] ***, Locust, <https://locust.io/>, ultima accesare: 30/05/2026.
- [35] ***, cryptography (Python library), <https://cryptography.io/en/latest/>, ultima accesare: 30/05/2026.
- [36] ***, SymPy, <https://www.sympy.org/>, ultima accesare: 30/05/2026.
- [37] R. Fielding, J. Reschke, "Hypertext Transfer Protocol (HTTP/1.1): Semantics and Content", RFC 7231, Internet Engineering Task Force, iun. 2014, <https://datatracker.ietf.org/doc/html/rfc7231>.
- [38] ***, OWASP Secure Headers Project, <https://owasp.org/www-project-secure-headers/>, ultima accesare: 30/05/2026.
- [39] D. Crocker, P. Overell, "Augmented BNF for Syntax Specifications: ABNF", RFC 5234, Internet Engineering Task Force, ian. 2008, <https://datatracker.ietf.org/doc/html/rfc5234>.
- [40] ***, OWASP Cross-Site Scripting (XSS) Prevention Cheat Sheet, https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Scripting_Prevention_Cheat_Sheet.html, ultima accesare: 30/05/2026.
- [41] D. Wong, "Real-World Cryptography", Manning Publications, 2021, cap. 12.
- [42] ***, MDN Web Docs: Cache-Control, <https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Cache-Control>, ultima accesare: 30/05/2026.
- [43] ***, MDN Web Docs: Crypto.getRandomValues(), <https://developer.mozilla.org/en-US/docs/Web/API/Crypto/getRandomValues>, ultima accesare: 30/05/2026.
- [44] C. Percival, S. Josefsson, "The scrypt Password-Based Key Derivation Function", RFC 7914, Internet Engineering Task Force, aug. 2016, <https://datatracker.ietf.org/doc/html/rfc7914>.
- [45] W. Diffie, M. Hellman, "New Directions in Cryptography", IEEE Transactions on Information Theory, vol. 22, nr. 6, pp. 644-654, nov. 1976, doi: 10.1109/TIT.1976.1055638.
- [46] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3", RFC 8446, Internet Engineering Task Force, aug. 2018, <https://www.rfc-editor.org/rfc/rfc8446>.
- [47] R. P. Fernandez et al., "Post-Quantum Threats to TLS: A Survey", Cryptography, vol. 9, nr. 4, art. 73, 2025, <https://www.mdpi.com/2410-387X/9/4/73>.
- [48] A. Papachristodoulou et al., "TLS 1.3 Traffic Analysis: Challenges and Insufficiencies", Electronics, vol. 13, nr. 20, art. 4000, 2024, <https://www.mdpi.com/2079-9292/13/20/4000>.

- [49] ***, What the Post-Quantum Shift Means for Your Security Strategy, TechRadar, <https://www.techradar.com/pro/what-the-post-quantum-shift-means-for-your-security-strategy>, ultima accesare: 30/05/2026.
- [50] D. J. Bernstein, T. Lange, "Post-Quantum Cryptography", Cryptology ePrint Archive, Report 2015/1075, 2015, <https://eprint.iacr.org/2015/1075.pdf>.
- [51] ***, OWASP Testing Guide: Session Management, https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/06-Session_Management_Testing/, ultima accesare: 30/05/2026.
- [52] C. Allen, "The Path to Self-Sovereign Identity", 2016, <http://www.lifewithalacrity.com/2016/04/the-path-to-self-sovereign-identity.html>.
- [53] ***, Nostr Community Group: HTTP Schnorr Authentication Draft, <https://nstrcg.github.io/http-schnorr-auth>, ultima accesare: 30/05/2026.
- [54] P. Kumar, "Efficacy of Schnorr Signature for Stateless Authentication", Medium, <https://medium.com/%40prathyusha756/efficacy-of-schnorr-signature-for-stateless-authentication-5ae5d65ec5d3>, ultima accesare: 30/05/2026.
- [55] T. Wu, "The Secure Remote Password Protocol (SRP)", RFC 5054, Internet Engineering Task Force, nov. 2007, <https://datatracker.ietf.org/doc/html/rfc5054>.
- [56] ***, PAKE: Password Authenticated Key Exchange, <https://asecuritysite.com/pake>, ultima accesare: 30/05/2026.
- [57] ***, Password Authenticated Key Exchange by Juggling (J-PAKE), Wikipedia, https://en.wikipedia.org/wiki/Password_Authenticated_Key_Exchange_by_Juggling, ultima accesare: 30/05/2026.
- [58] ***, SRP: A Zero-Knowledge Protocol for Password Authentication, Medium, <https://medium.com/@mainnetready/srp-a-zero-knowledge-protocol-for-password-authentication-1e19582aab29>, ultima accesare: 30/05/2026.
- [59] V. Shah et al., "Survey on Zero-Knowledge Proof Authentication Protocols", arXiv:2401.11735, 2024, <https://arxiv.org/abs/2401.11735>.
- [60] ***, Zero Knowledge Authentication, Sedicii, <https://sedicii.com/news/zero-knowledge-authentication/>, ultima accesare: 30/05/2026.
- [61] ***, Meta Engineering Blog: Post-Quantum Readiness for TLS, <https://engineering.fb.com/2024/05/22/security/post-quantum-readiness-tls-pqr-meta/>, ultima accesare: 30/05/2026.
- [62] ***, Cloudflare Blog: A Primer on Lattice Cryptography, <https://blog.cloudflare.com/lattice-crypto-primer/>, ultima accesare: 30/05/2026.
- [63] A. Z. Vyas et al., "Side-Channel Vulnerabilities of Post-Quantum Cryptographic Algorithms", DIVA Portal, 2023, <https://www.diva-portal.org/smash/get/diva2%3A1742628/FULLTEXT01.pdf>.
- [64] F. Deng et al., "Real-World Implementation Weaknesses and Attack Risks in TLS", Computers and Security, 2025, <https://www.sciencedirect.com/science/article/abs/pii/S1574013725000140>.

Anexe

Anexa 1. Ssd

A1. ex:

- ex

A.1. Arhitectura aplicației.

Anexa 2. Sesd

Codul ex

.